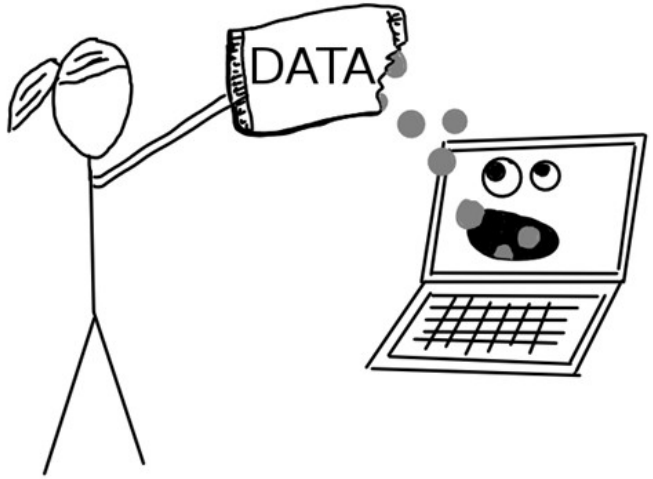
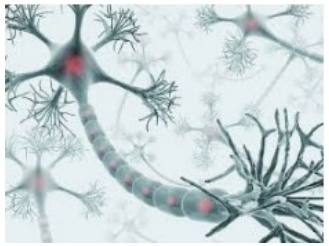
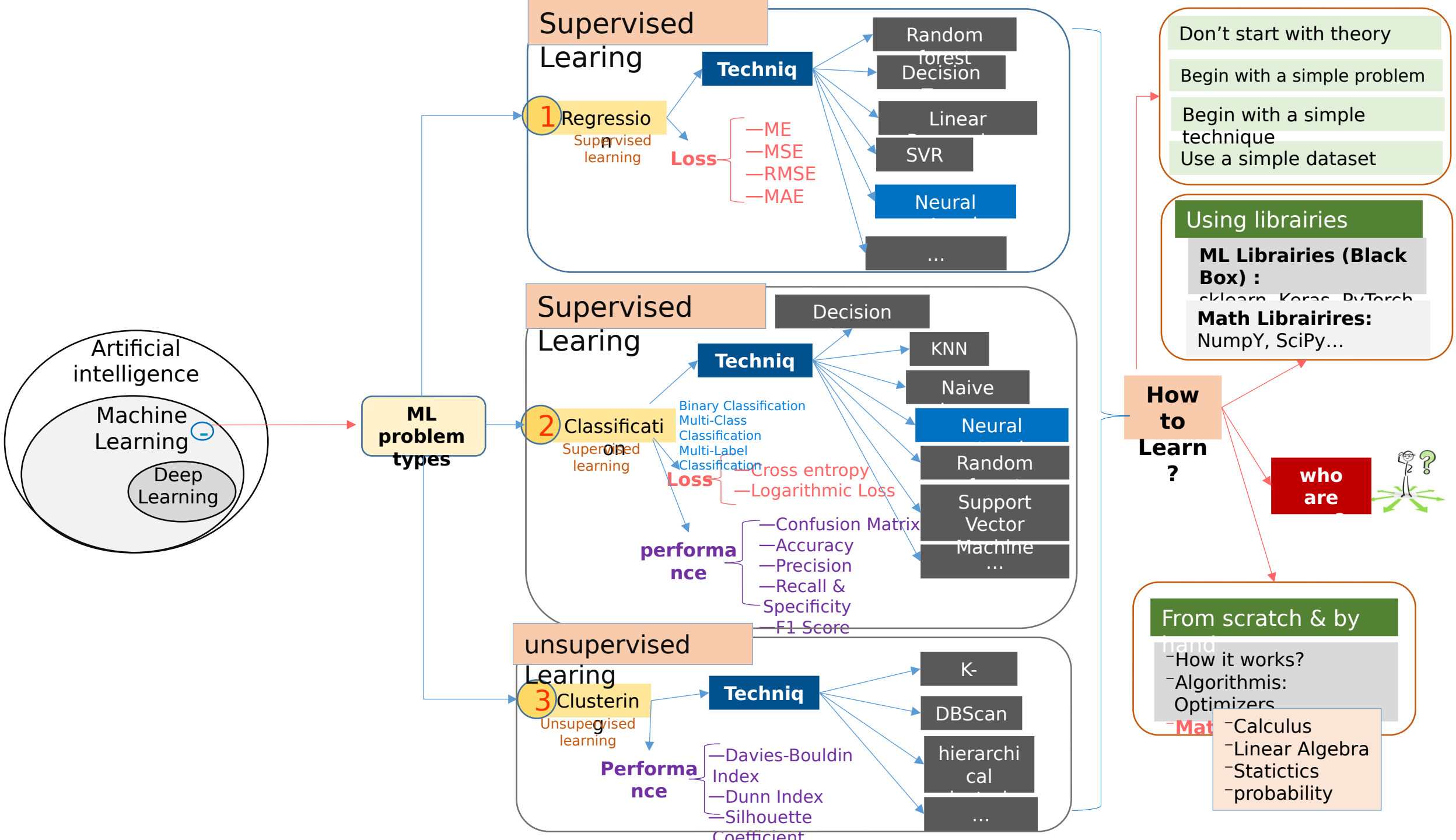




Linear Regression
Logistic Regression
Neural Network

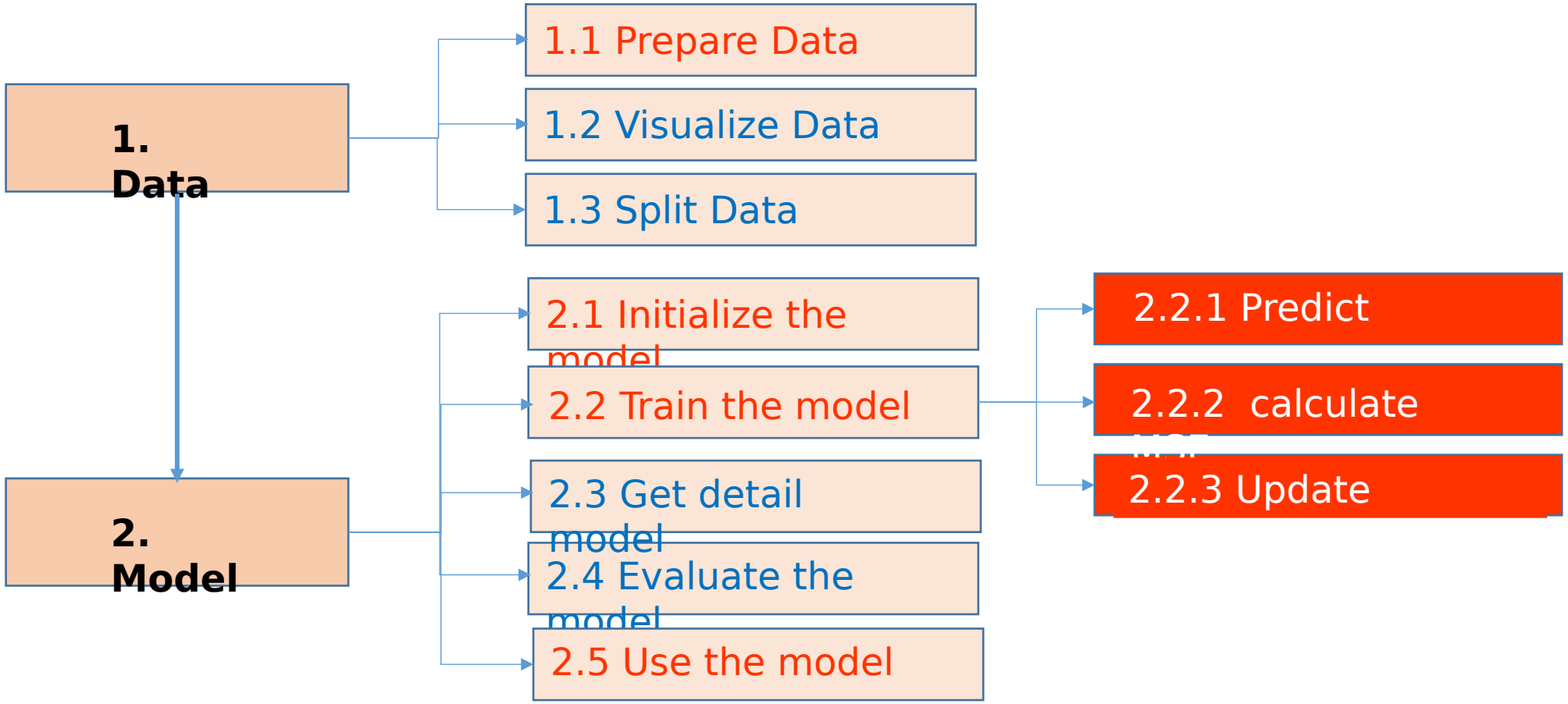
2022/20

Abdelwahab Naji - Enseignant chercheur
Abdelwahab.naji@gmail.com
ENSAO, Université Mohamed premier - Oujda
https://www.youtube.com/c/AbdelwahabNaji


Artificial intelligence
Machine Learning
Deep Learning
ML problem types
Supervised Learing
1 Regression
Supervised learning
Techniq
Loss
—ME
—MSE
—RMSE
—MAE
Random forest
Decision
Linear
SVR
Neural
...
Supervised Learing
2 Classification
Supervised learning
Techniq
Binary Classification
Multi-Class Classification
Multi-Label Classification
Loss
—Cross entropy
—Logarithmic Loss
performance
—Confusion Matrix
—Accuracy
—Precision
—Recall & Specificity
—F1 Score
Decision
KNN
Naive
Neural
Random
Support Vector Machine
...
unsupervised Learing
3 Clustering
Unsupervised learning
Techniq
Performance
—Davies-Bouldin Index
—Dunn Index
—Silhouette Coefficient
K-
DBScan
hierarchical
...
How to Learn ?
Don't start with theory
Begin with a simple problem
Begin with a simple technique
Use a simple dataset
Using librairies
ML Librairies (Black Box) :
sklearn, Keras, PyTorch
Math Librairires:
NumpY, SciPy...
who are
From scratch & by hand
-How it works?
-Algorithmis: Optimizers
-Mat
-Calculus
-Linear Algebra
-Statictics
-probability

Machine Learning

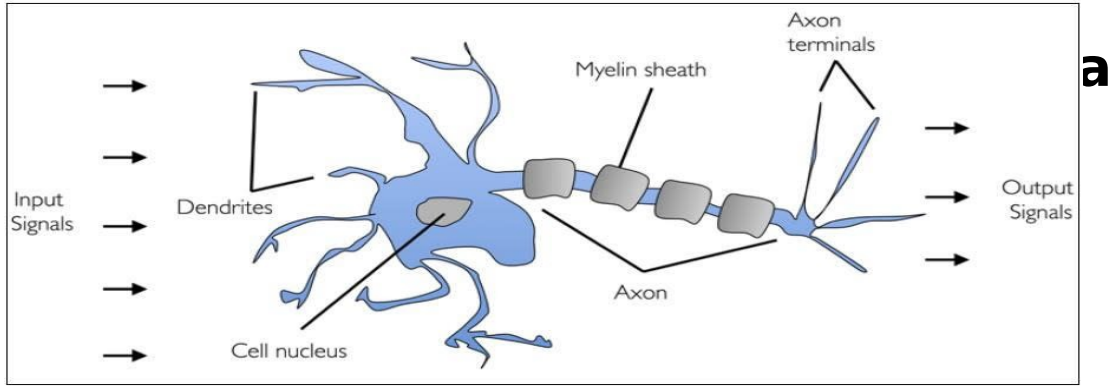
Les éléments + étapes



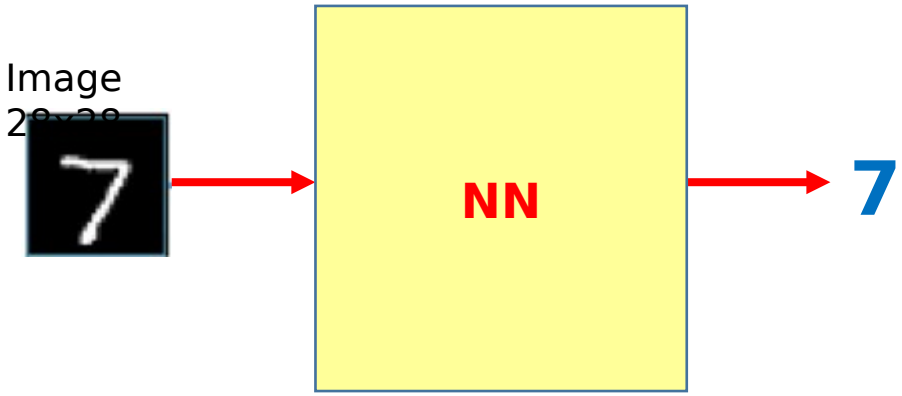
Biological Neuron

- **Un humain a des billions de neurons**
- **Les neurones sont des cellules nerveuses interconnectées dans le cerveau humain qui participent au traitement et à la transmission de signaux chimiques et électriques.**
- **Les dendrites sont des branches qui reçoivent (in) des informations d'autres neurones.**
- **Le noyau cellulaire ou Soma traite les informations reçues des dendrites.**
- **Axon est un câble utilisé par les neurones pour envoyer des informations (out).**
- **La synapse est la connexion entre un ales.**

Image
28x28



Human



Computer

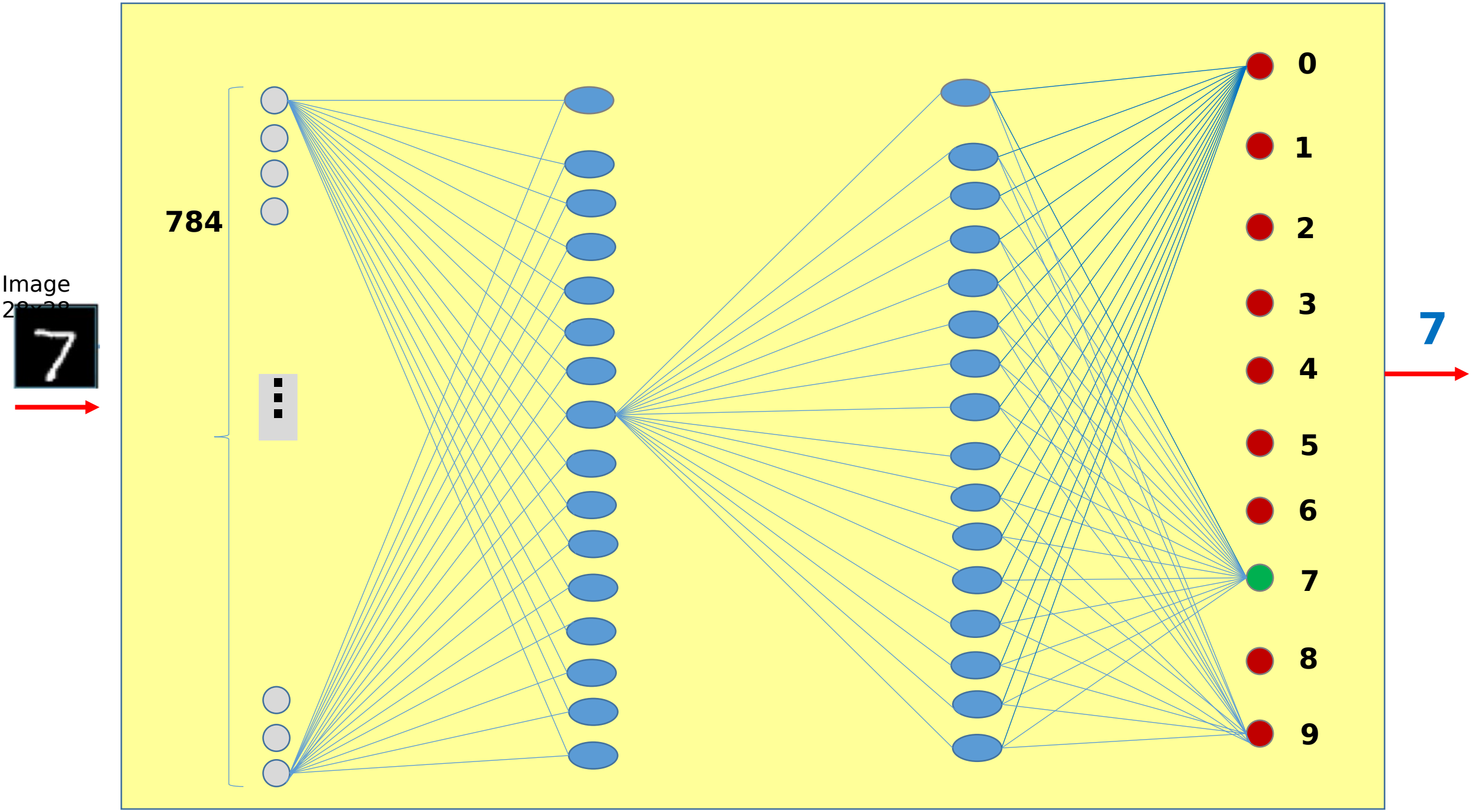
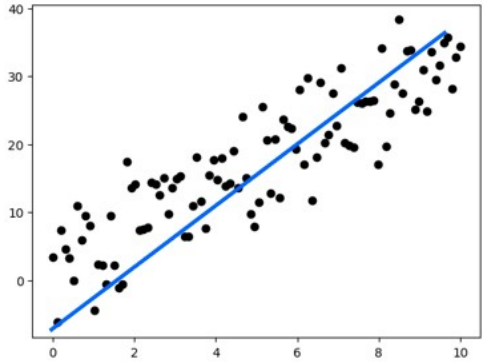


Image
28x28
784
0
1
2
3
4
5
6
7
8
9
7

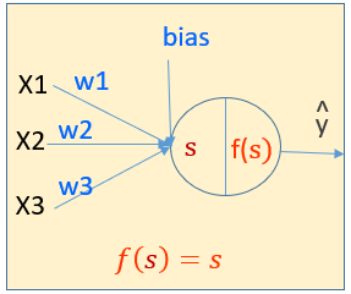
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

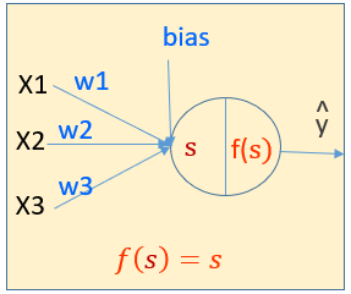


2

Multi Variate Linear Regression Problem

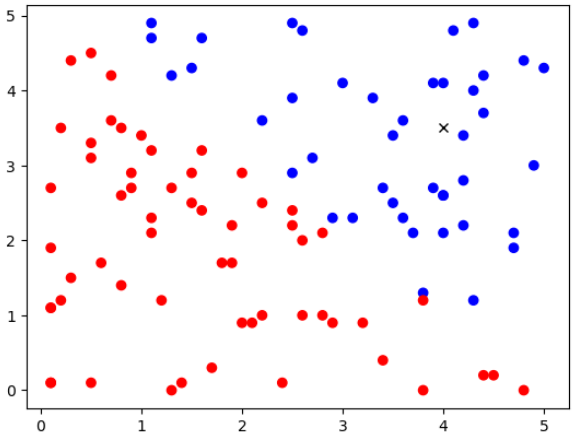
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising p

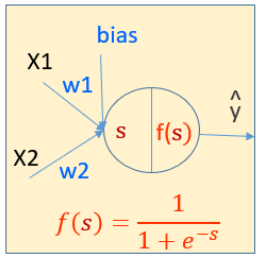


3

Linearly Separable problem

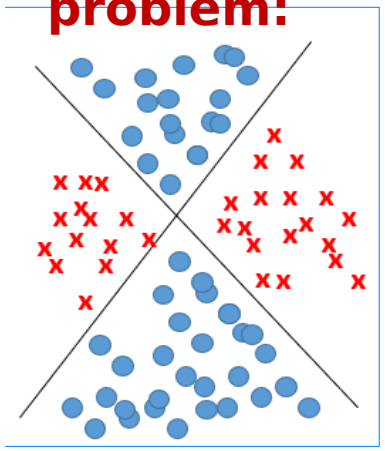


Red or Blue?

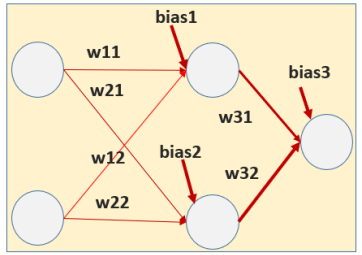


4

Non Linearly Separable problem:

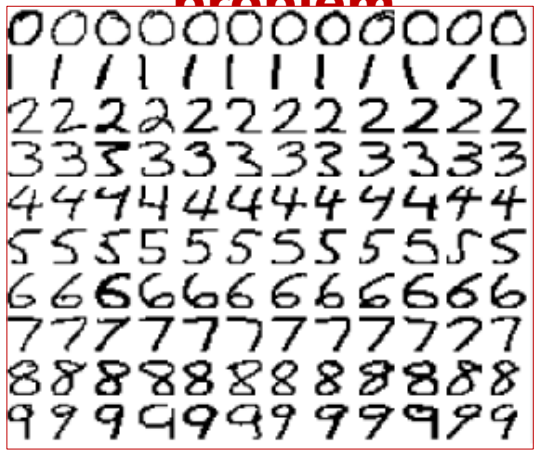


XOR Problem

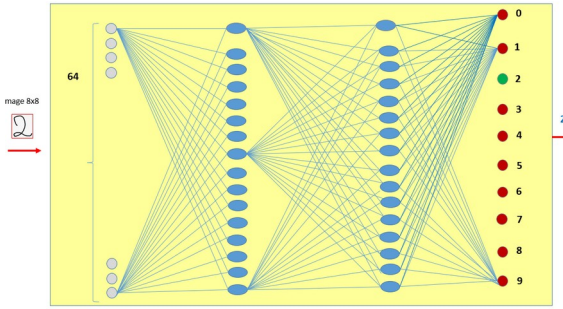


5

Complex classification problem



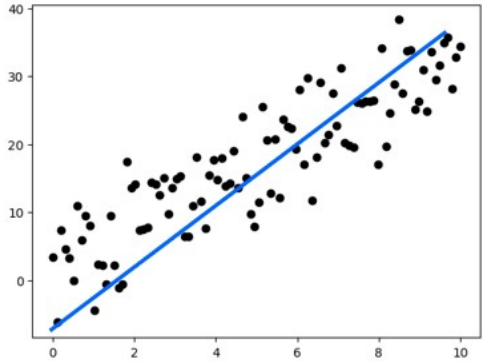
Hand Written digits



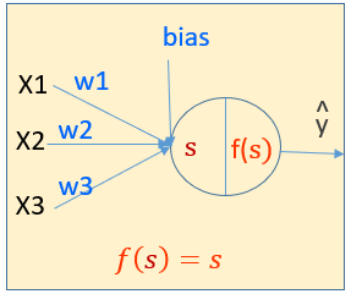
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

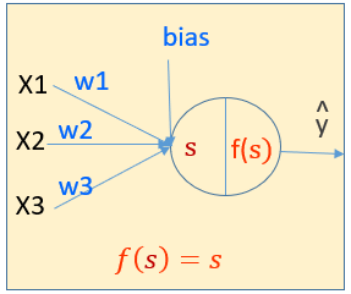


2

Multi Variate Linear Regression Problem

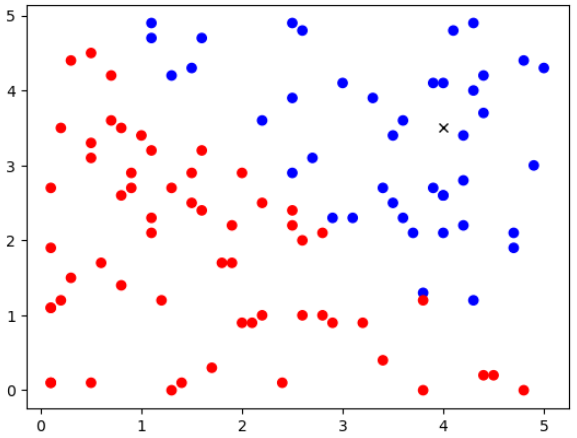
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising p

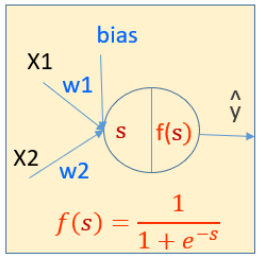


3

Linearly Separable problem

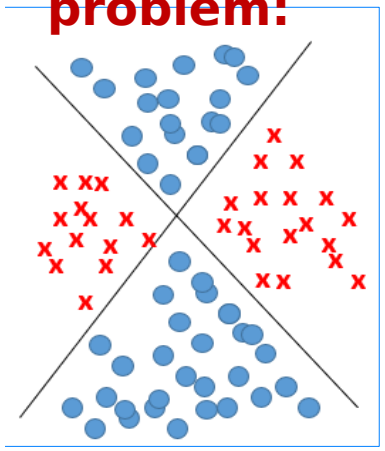


Red or Blue?

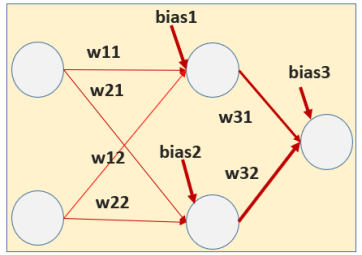


4

Non Linearly Separable problem:

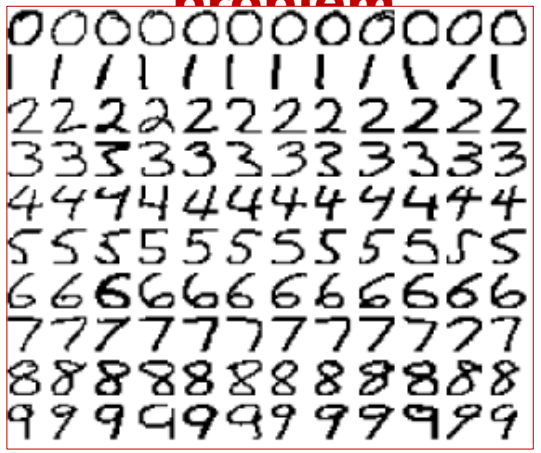


XOR Problem

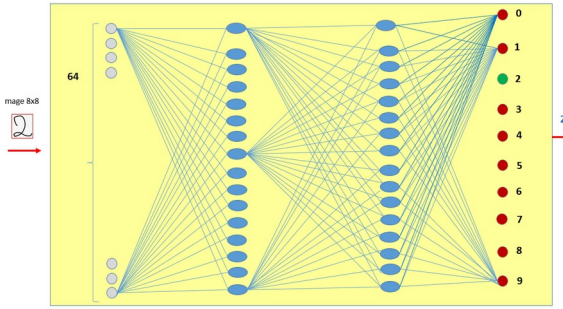


5

Complex classification problem

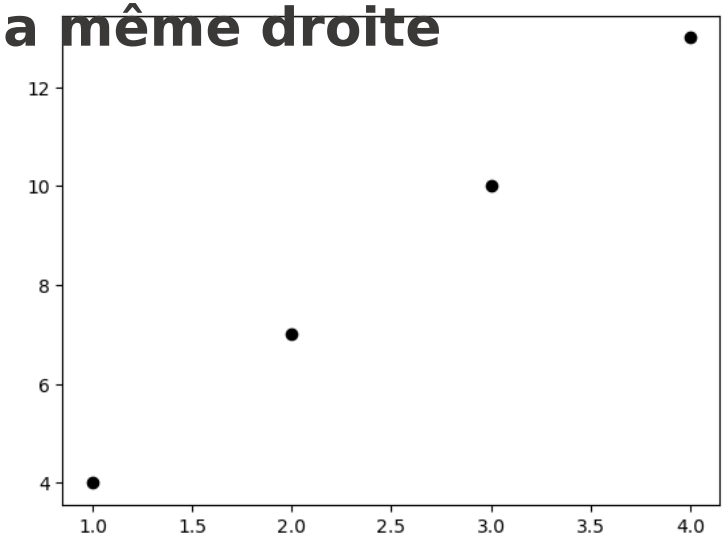


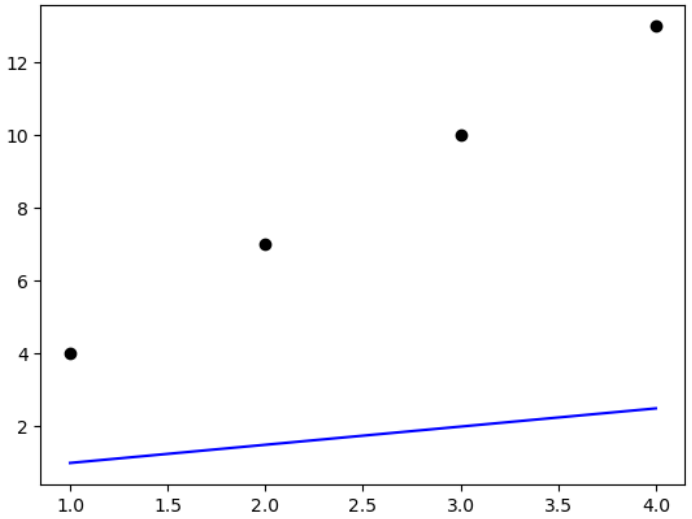
Hand Written digits

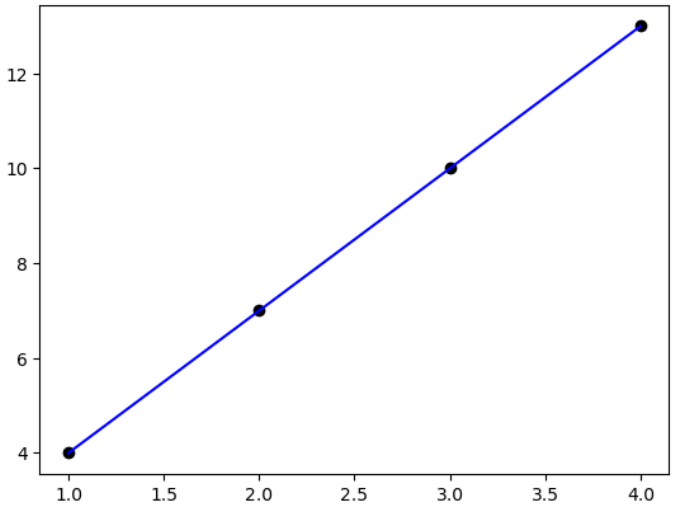


Un modèle de régression

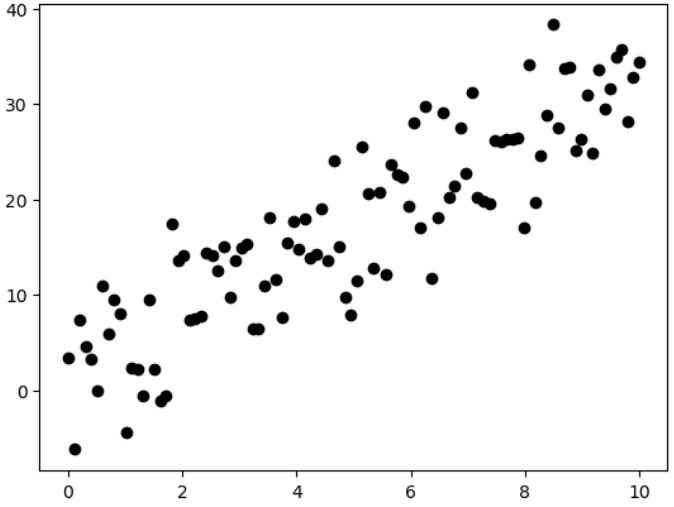
Cas 1: points (Historique) se trouvant sur la même droite

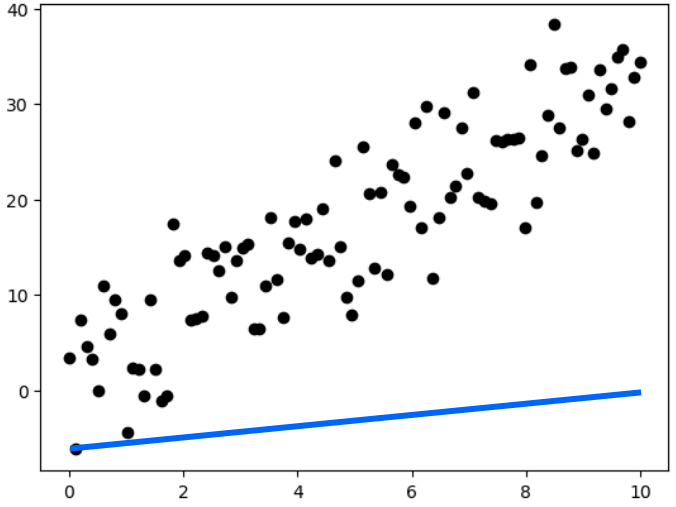


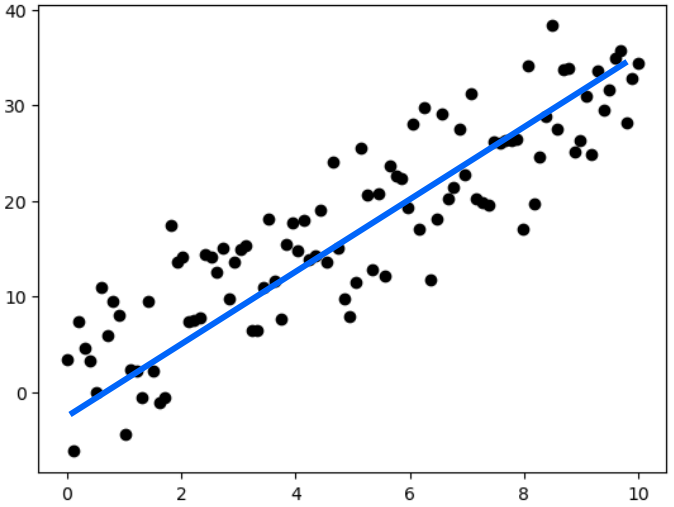




Cas 2: nuage de points

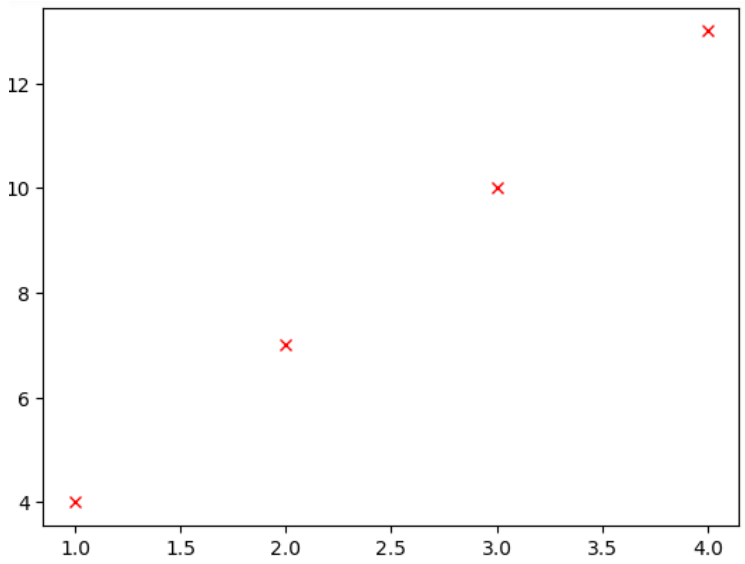


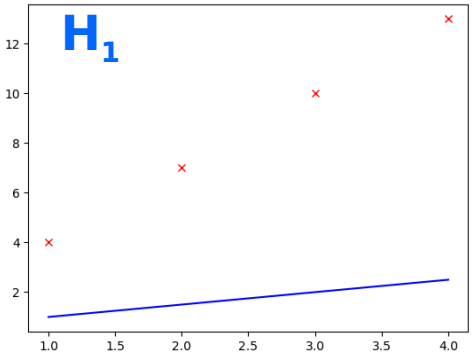


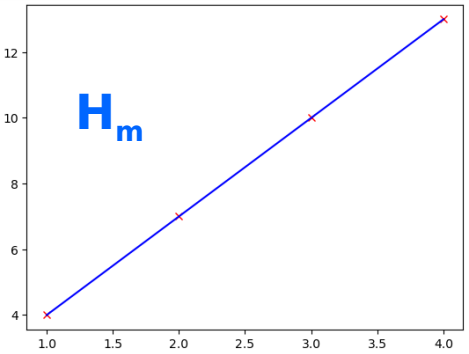


Un modèle de régression

Data	
X1	Y
1	4
2	7
3	10
4	13
7	22







Objectif:

Trouver un modèle ayant la forme:

$H(X1) = bias + w_1 * X1$

Exemple 1 de modèle

$H(X1) = 1 + 1 * X1$

Tester le modèle:

$H(7) = 1 + 1 * 7 = 8$

predicted_y=8
actual_y=22
error= actual_y-predicted_y=-14

Exemple 2 de modèle

$H(X1) = 1 + 3 * X1$

Tester le modèle:

$H(7) = 1 + 3 * 7 = 22$

predicted_y=22
actual_y=22
error= actual_y-predicted_y=0

Trouver le bon modèle: Apprentissage

- L'apprentissage se fait sur le dataset
- Le bon modèle minimise une fonction objective: mean quared errors

Exemple

- Soit le modèle H1(X)=1+1*X
- Calculer predicted_y,error, squared_error, mean_squared_error

X1	Y
1	4
2	7
3	10
4	13

Predicted_Y
2
3
4
5

error
2
4
6
8

Squared_error
4
16
36
64

mse=30

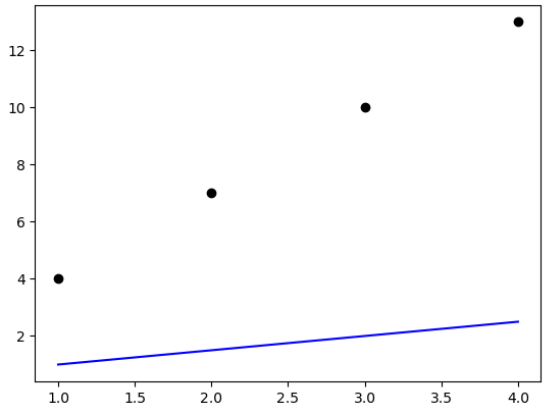
- On a une infinité de modèle (chaque combinaison de bias et de w1)
- Le bon modèle c'est lui qui minimise mse

Trouver le bon modèle: Apprentissage

$H(X1) = bias + w_1 * X1$

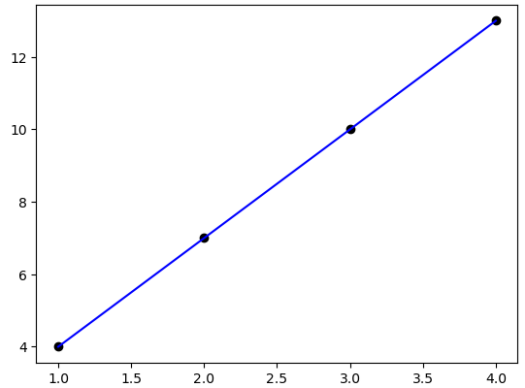
Pour trouver le modèle, on suit l'algorithme suivant

```
Apprentissage:
  debut
   initialiser modèle (bias,w)
   répéter epochs fois:
     prédire en utilisant le modèle
     connaitre error, squared error,mean squared_error
     corriger le modèle (bias,w)
fin
```



$H(X1) = 0 + 1 * X1$

$H(X1) = 1 + 3 * X1$



L’algorithme de gradient descent

Gradient Descent Optimization Algorithm

Inputs: X, Y

lr=0.01? **epochs=10000?**

1. **Initialize bias and weights (W)**
   w1=0.5, bias=0.5

For i=0 to epochs do

2. **Predict**
   $\hat{y}$=**bias** + **w1***x

3. **Minimize the Mean Squared Error ?**

$$mse(bias, w1) = \frac{1}{n}\sum_{i=1}^{n}(y_i - (bias + w1 * xi))^2$$

dw1

$$\frac{\partial mse}{\partial w1} = \frac{2}{n}\sum_{1}^{n} -xi(yi - (bias + w1 * xi))$$

dbias

$$\frac{\partial mse}{\partial bias} = \frac{2}{n}\sum_{1}^{n} -(yi - (bias + w1 * xi))$$

4. **Update bias and W**
   bias=bias-dbias*lr
   w1=w1-dw1*lr

End for

5. **Return w1, bias**

Minimiser MSE pour avoir le bon modèle. Comment?

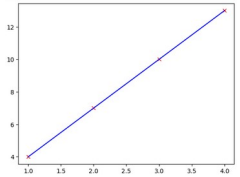
On suppose que bias=1, comment Mean Square Error (MSE) varie en fonction de w1

yhat=h(X1)= bias+w_1 * X1

$$ms\mathcal{E} = \frac{1}{n}\sum_{i=1}^{n}(y_i - yhat_i)^2$$

Data

X1	Y
1	4
2	7
3	10
4	13



Apprentissage:
debut
initialiser modèle (bias,w)
répéter epochs fois:
prédire en utilisant le modèle
connaitre l'erreur
corriger le modèle (bias,w)
fin

```
def fit(X,y):
    n=len(y)
    w1=0
    bias=1
    epochs=60
    for k in range(epochs):
        yhat=bias+w1*X
        mse=(1/n)*np.sum((y-yhat)**2)
        w1=w1+0.1
    return w1
```

Algorithme naif

Exécution pas à pas du programme:

Itération 0: w1=0

Y	$\hat{Y}$	e error	e^2 Squared_error
4	1	3	9
7	1	6	36
10	1	9	81
13	1	12	144

MSE 67.5

Itération 1 : w1=0.1

Y	$\hat{Y}$	e error	e^2 Squared_error
4	1.1	2.9	8.41
7	1.2	5.8	33.64
10	1.3	8.7	75.69
13	1.4	11.6	134.56

MSE 63.07

It 10 :w1=1

$\hat{Y}$
2
3
4
5

MSE 30

It 29:w1=2.9

$\hat{Y}$
3.6
6.5
9.7
12.6

MSE 0.075

It 30:w1=3

$\hat{Y}$
4
7
10
13

MSE 0

It 31:w1=3.1

$\hat{Y}$
4.1
7.2
10.3
13.4

MSE 0.075

It 59 :w1=5.9

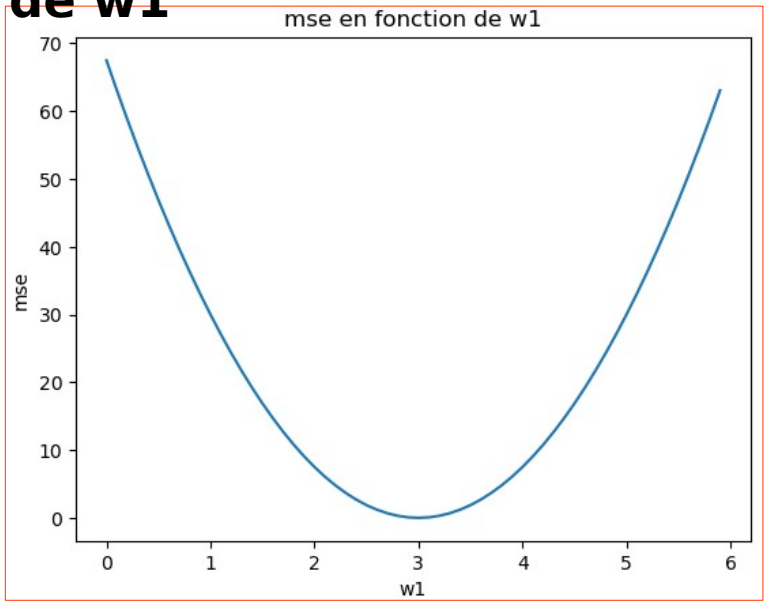
$\hat{Y}$
6.9
12.8
18.7
24.6

MSE 63.07

Interprétation graphique

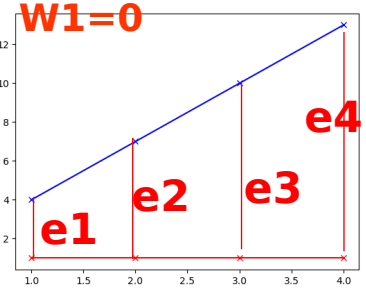
```
def fit(X,y):
    w1=0
    bias=1
    epochs=60
    n=len(y)
    for k in range(epochs):
        yhat=bias+w1*X
        mse=(1/n)*np.sum((y-yhat)**2)
        w1=w1+0.1
    return w1
```

Évolution de mse en fonction de w1

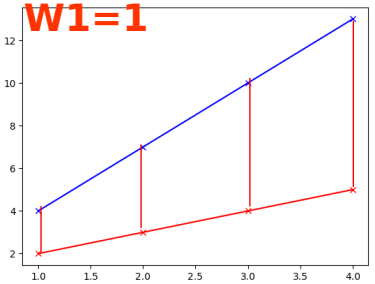


Les mises à jour du modèle

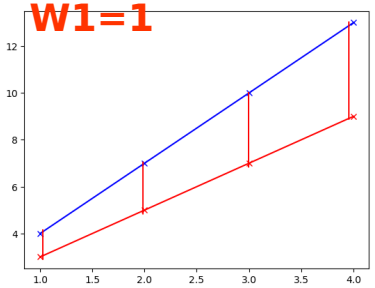
Iteration 0, W1=0



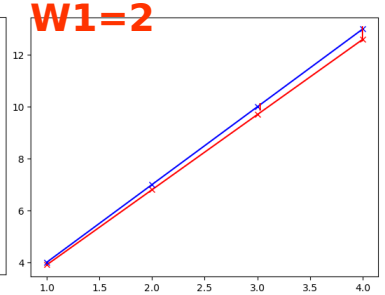
Iteration 10, W1=1



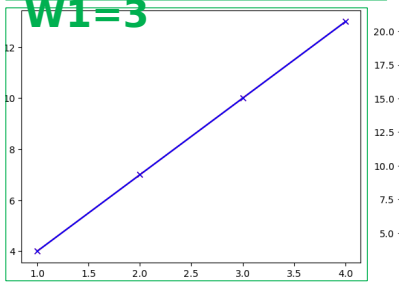
Iteration 10, W1=1



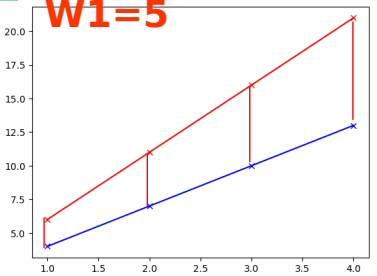
Iteration 20, W1=2



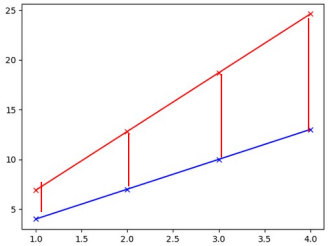
Iteration 30, W1=3



Iteration 50, W1=5



Iteration 59, W1=5.9



Problème d’orientation de la mise à jour

- Pour avoir le w1 optimal:
  - On ne peut pas connaitre la bonne valeur de w1 initiale: Orientation
    - W1=w1 +/- dw
  - Problème de step (valeur de mise à jour)
    - W1=w1 – dw (dw est très petit ou grand)

Utiliser Gradient Descent

```
lr=0.01
def fit(X,y):
    n=len(y)
    w1=0
    bias=0
    for k in range(1
        yhat=bias+w1

        dbias=-(2/n)*np.sum(y-yhat)
        dw1=-(2/n)*np.sum(X*(y-yhat))
        bias=bias-dbias*lr
        w1=w1-dw1*lr
    return bias,w1
```

initialize the model

Predict using the

Update the model

Le même algorithme marche pour un nuage de points

```
# 100 abscisses entre 0 et 10
X=np.linspace(0,10,100)
y=2+3*X     # y à générer sur une droite
 # valeurs aléatoires à ajouter à y
r=np.random.uniform(-7, 7, size=len(y))
y=y+r # update de y autour de la droite
plt.scatter(X,y,color="black")
```

```
lr=0.01
def fit(X,y):
    n=len(y)
    w1=0
    bias=0
    for k in range(1
        yhat=bias+w
        dbias=-(2/n)*np.sum(y-yhat)
        dw1=-(2/n)*np.sum(X*(y-yhat))
        bias=bias-dbias*lr
        w1=w1-dw1*lr
    return bias,w1
```

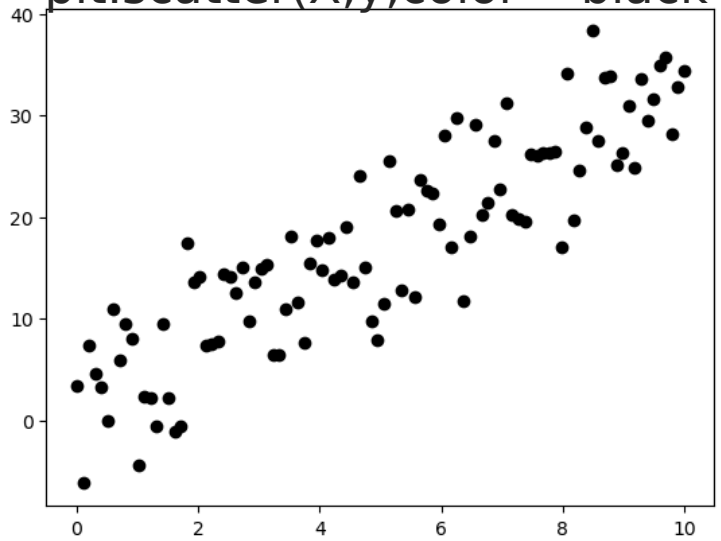
initialize the model

Predict using the

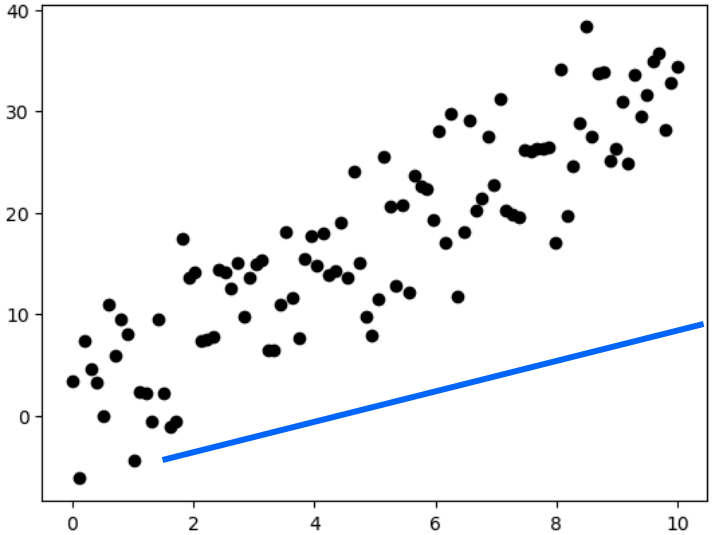
Update the model

```
bias,w1=fit(X,y)
#utiliser le modèle
def predict(X):
    return bias+w1*X
#tracer le nuage de points
plt.plot(X,y,"o",color="black")
#tracer le modèle (droite)
abscisses=np.array([0,10])
ordonnees=model(absci
```

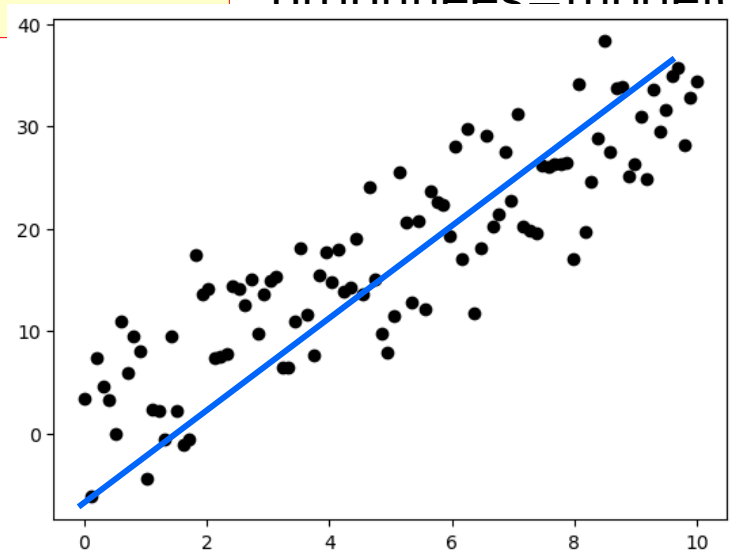
don



Nuage de points

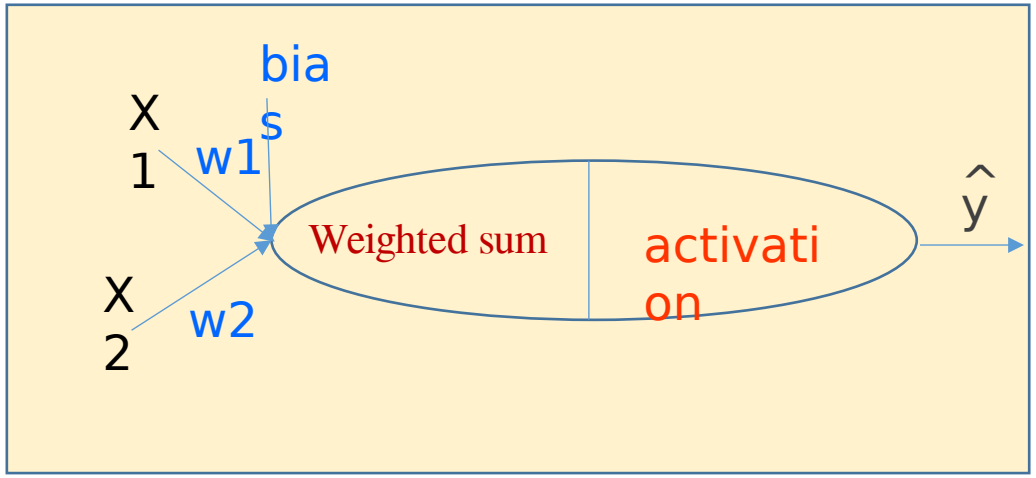


Modèle initial: bias= , w1=



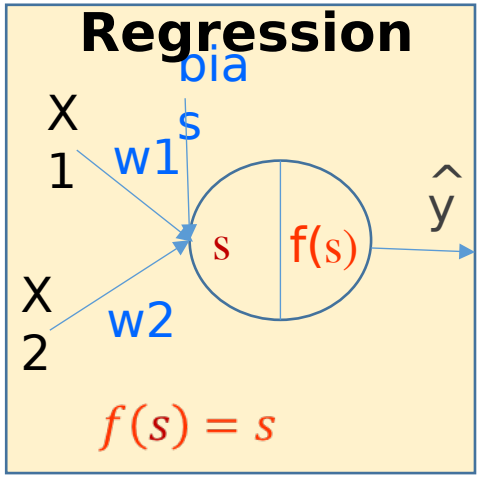
Modèle après apprentissage:

Réutiliser le même code source pour créer un réseau de neurone (un seule neurone)



Regression

Linear Regression



$$weighted\ sum = bias + w1 * X1 + w2 * X2$$

Activation: une fonction d'activation

- Identity or linear
- Sigmoid
- Tanh
- Relu
- Leaky ReLU
- ...

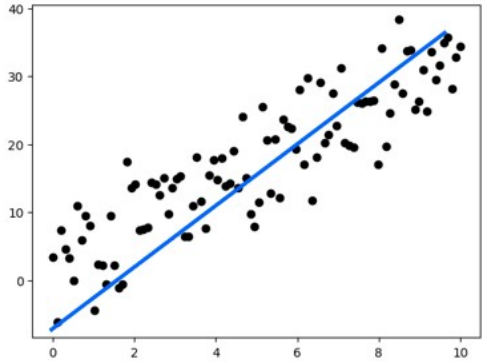
Le choix de la fonction d'activation dépend de...

Code source

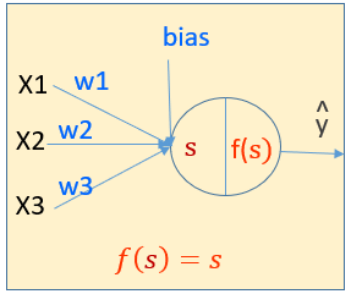
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

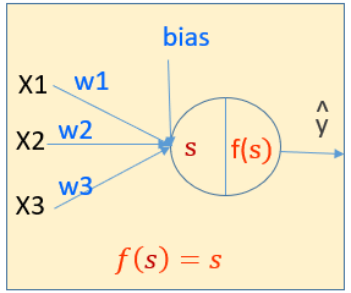


2

Multi Variate Linear Regression Problem

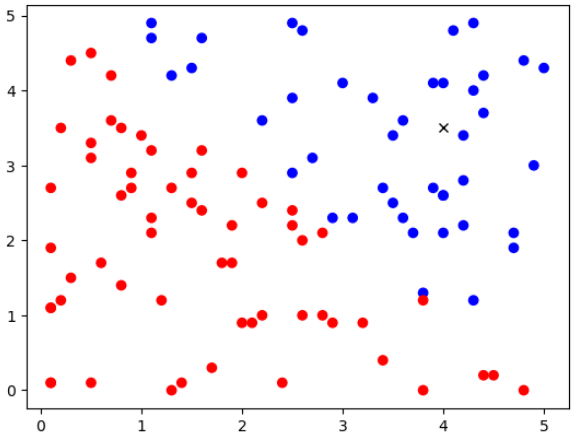
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising p

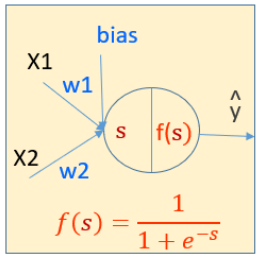


3

Linearly Separable problem

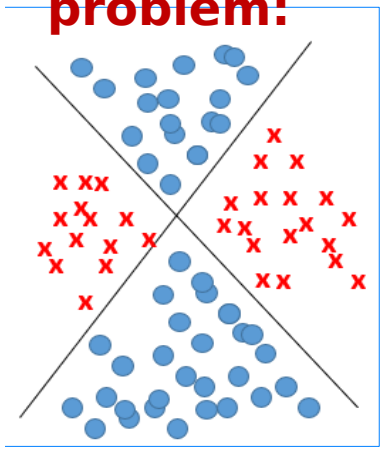


Red or Blue?

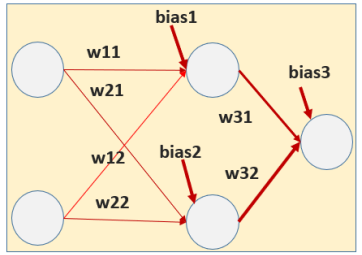


4

Non Linearly Separable problem:

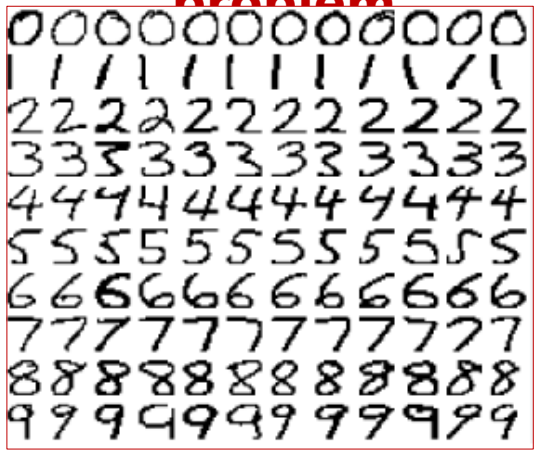


XOR Problem

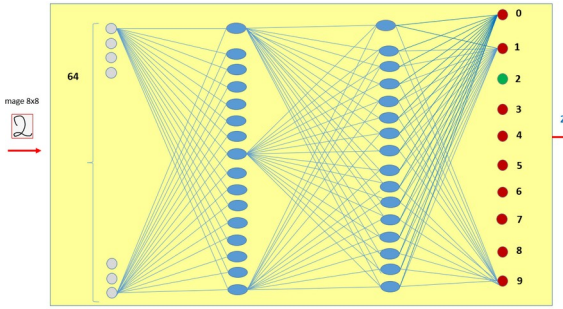


5

Complex classification problem



Hand Written digits



Exemple

- Une société souhaite analyser le chiffre réalisé (**Sales**) en fonction du budget alloué à la publicité (frais)
- La publicité se fait en utilisant différents plateformes média (TV, Radio, newspaper)
- **Objectif:** prédire le total des ventes (sales) selon les frais de publicité

Data Set

TV	Radio	Np	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
151.5	41.3	58.5	**18,5**
180.8	10.8	58.4	**12,9**
8.7	48.9	75	**7,2**
57.5	32.8	23.5	**11,8**
120.2	19.6	11.6	**13,2**
8.6	2.1	1	**4,8**
199.8	2.6	21.2	**10,6**

X1=[230.1,37.8,69.2]

y1=22.1

Sales à prédire

TV	Radio	Newspaper	**Sales** (target)
66.1	5.8	24.2	?
214.7	24	4	?
23.8	35.1	65.9	?

- Dans le Dataset chaque exemple d'apprentissage est accompagné par une target: Supervised Learning (Apprentissage supervisé)
- Objectif: Prédire une valeur continue
problème de régression

- (X_i, y_i): un exemple d'apprentissage
- features: Tv, Radio, newsPaper (X)
- target: sales (y)

Sales_predicted = bias+w_1*X1+W_2*X2+ W_3*X3

https://www.kaggle.com/ashydv/advertising-dataset

Si les valeurs de X sont grandes, afin d'éviter des mises à jours des poids très grandes, il faut standardiser les données

```
from sklearn.preprocessing import StandardScaler
scaler= StandardScaler()
X_st = scaler.fit_transform(X)
```

$$z = \frac{x - \mu}{\sigma}$$

- z: variable standardisée
- μ: Moyenne
- σ: Déviation standard

1 feature, le modèle est: bias+w1*X1

```
lr=0.01
def fit(X,y):
    n=len(y)
    w1=0
    bias=0
    for k in range(1000):
        yhat=bias+w
        dbias=-(2/n)*np.sum(y-yhat)
        dw1=-(2/n)*np.sum(X*(y-yhat))
        bias=bias-dbias*lr
        w1=w1-dw1*lr
```

initialize the model

Predict using the model

Update the model

```
def predict(X1):

    return bias+w1*X1

#penser à la remise à l'échelle si X1 est standardisées
```

3 features, le modèle est: bias+w1*X1 +w2*X2 +w3*X3

```
lr=0.002
def fit(X1,X2,X3,y):
    n=len(y)
    w1=0
    w2=0
    w3=0
    bias=0
    for k in range(10000):
        yhat=bias+w1*X1+w2*X2+w3*X3

        dbias=-(2/n)*np.sum(y-yhat)

        dw1=-(2/n)*np.sum(X1*(y-yhat))
        dw2=-(2/n)*np.sum(X2*(y-yhat))
        dw3=-(2/n)*np.sum(X3*(y-yhat))

        bias=bias-dbias*lr

        w1=w1-dw1*
        w2=w2-dw2*
        w3=w3-dw3*
    return bias,w1,w2,w3
```

initialize the model

Predict using the model

Update the model

X1, X2 et X3 doivent être standardisés

```
def predict(X1,X2,X3):

    return
bias+w1*X1+w2*X2+w3*X3

#penser à la remise à l'échelle si X1,X2,X3 sont standardisées
```

m features, le modèle est: bias+w1*X1 +w2*X2 +w3*X3+… +wm*Xm

X a m features
Et si on veut que fit soit capable de traiter m features!!!

```
lr=0.001
def fit(X,y):
    n=len(y)
    w1=0
    w2=0
    w3=0
    bias=0
    for k in range(10000):
        yhat=bias+w1*X1+w2*X2+w3*X3
        dbias=-(2/n)*np.sum(y-yhat)

        dw1=-(2/n)*np.sum(X1*(y-yhat))
        dw2=-(2/n)*np.sum(X2*(y-yhat))
        dw3=-(2/n)*np.sum(X3*(y-yhat))

        bias=bias-dbias*lr

        w1=w1-dw1*lr
        w2=w2-dw2*lr
        w3=w3-dw3*lr
    return bias,w1,w2,w3
```

Initialize the model

```
W=np.array([0,
W=np.array(np.zeros(X.shape[1]))
```

Predict using the model

```
yhat=bias+np.dot(X,W)
```

Update the model

```
dW=-(2/n)* np.dot(X.T,(y-yhat))
W=W-dW*lr
```

```
lr=0.001
def fit(X,y):
    n=len(y)
    #W=np.array([0,0,0])
    W=np.array(np.zeros(X.sha
     # X.shape[1] est le nombre d'attributs
    bias=0
    for k in range(10000):
        yhat=bias+np.dot(X,W)

         dbias=-(2/n)*np.sum(y-
         dW=-(2/n)* np.dot(X.T,(y-yhat))
        bias=bias-dbias*lr
         W=W-dW*lr

    return bias,W
```

initialize the model

Predict using the model

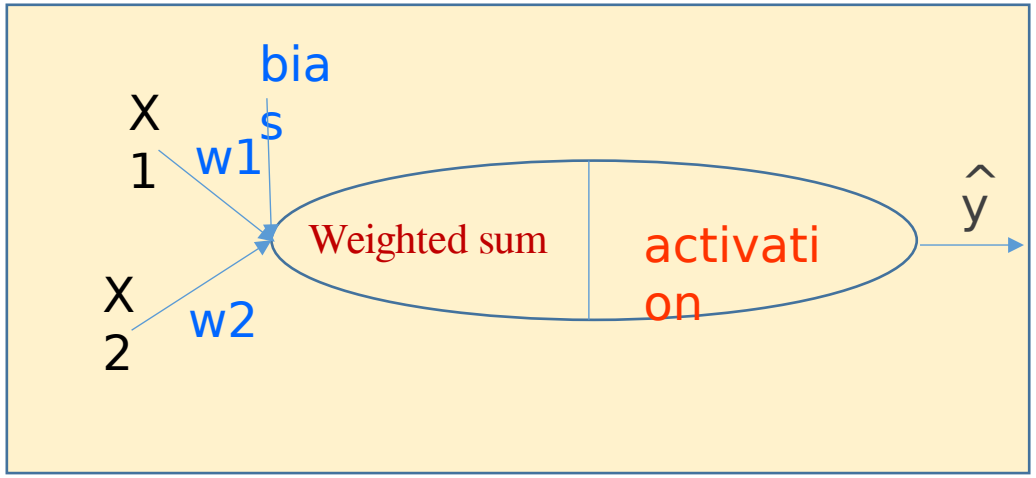
Update the model

```
def predict(X):

    return
bias+np.dot(X,W)
```

Si X1, X2 et X3 ont été standardisés,

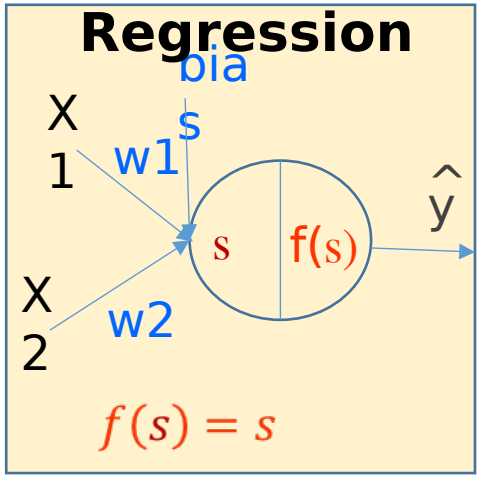
il faut les rendre à leurs échelles

Réutiliser le même code source pour créer un réseau de neurone (un seule neurone)



Regression

Linear Regression



$$weighted\ sum = bias + w1 * X1 + w2 * X2$$

Activation: une fonction d'activation

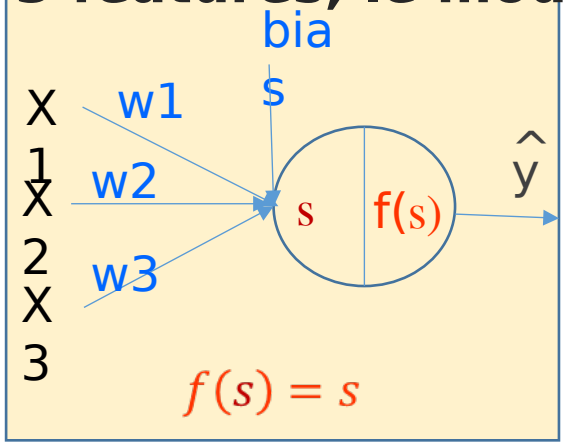
- Identity or linear
- Sigmoid
- Tanh
- Relu
- Leaky ReLU
- ...

Le choix de la fonction d'activation dépend de...

Code source

Regression Problem by using neural network (1 neurone)

3 features, le modèle est:



$weighted\ sum = bias + w1 * X1 + w2 * X2 + w3 * X3$

Activation: Identity or linear

```
def identity(z):
    return z
```

```
def predict(X1,X2,X3):

ws=bias+w1*X1+w2*X2+
w3*X3

    yhat=identity(ws)
```

```
lr=0.001
def fit(X1,X2,X3,y):
    n=len(y)
    w1=0
    w2=0
    w3=0
    bias=0
    for k in range(10000):
        ws=bias+w1*X1+w2*X2
        yhat=identity(ws)

        dbias=-(2/n)*np.sum(y-yhat)

        dw1=-(2/n)*np.sum(X1*(y-yhat))
        dw2=-(2/n)*np.sum(X2*(y-yhat))
        dw3=-(2/n)*np.sum(X3*(y-yhat))

        bias=bias-dbias*lr

        w1=w1-dw1*lr
        w2=w2-dw2*lr
        w3=w3-dw3*lr
    return bias,w1,w2,w3
```

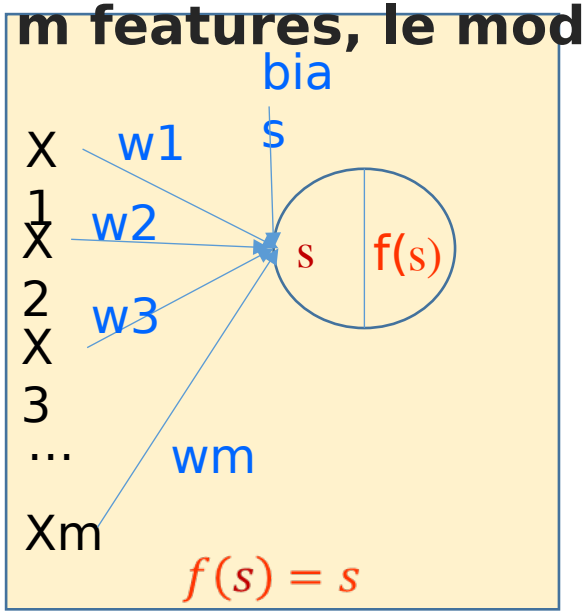
initialize the model

Predict using the model

Update the model

Regression Problem by using neural network (1 neurone)

m features, le modèle est:



$$weighted\ sum = bias + w1 * X1 + \cdots + wm * Xm$$

Activation: Identity or linear

```
def identity(z):
    return z
```

```
def predict(X):
    ws=bias+np.dot(X,W)
    yhat=identity(ws)
    return yhat
```

```
lr=0.001
def fit(X,y):
    n=len(y)
    #W=np.array([0,0,0])
    W=np.array(np.zeros(X.sha
     # X.shape[1] est le nombre d'attributs
    bias=0
    for k in range(10000):
        ws=bias+np.dot(X,W)
        yhat=identity(ws)

        dbias=-(2/n)*np.sum(y-yhat)
        dW=-(2/n)* np.dot(X.T,(y-yhat))
        bias=bias-dbias*lr
        W=W-dW*lr

    return bias,W
```

initialize the model

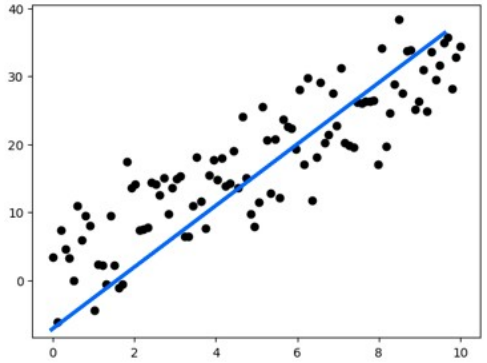
Predict using the model

Update the model

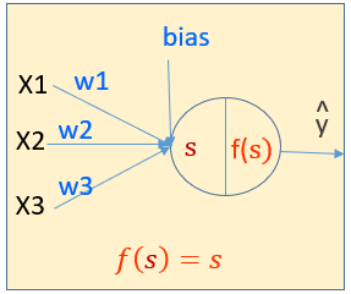
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

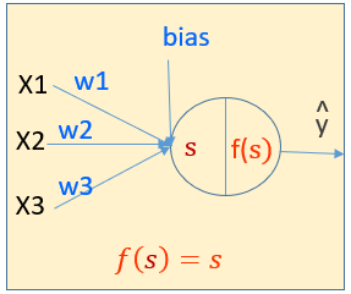


2

Multi Variate Linear Regression Problem

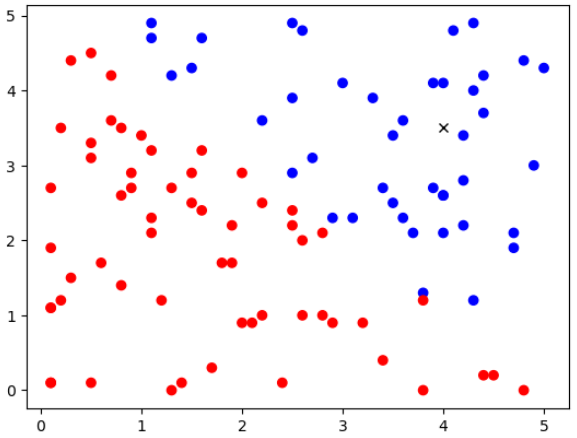
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising p

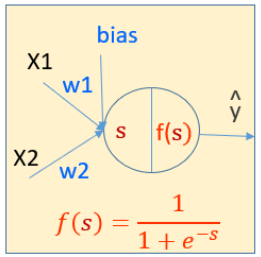


3

Linearly Separable problem

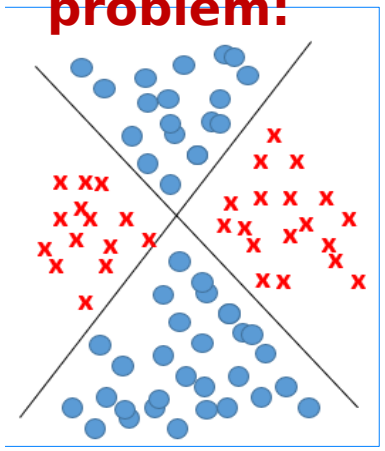


Red or Blue?

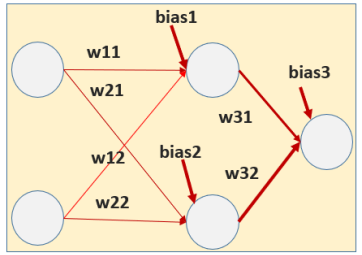


4

Non Linearly Separable problem:

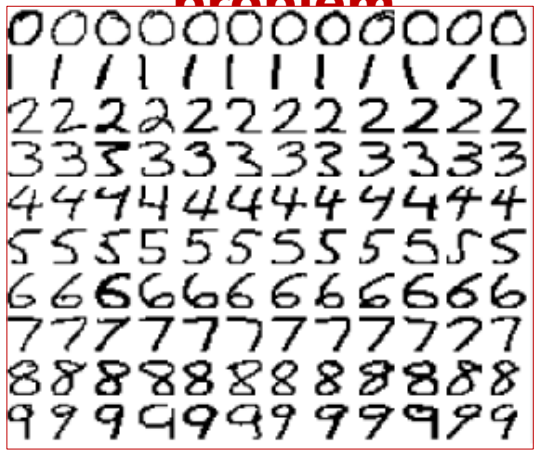


XOR Problem

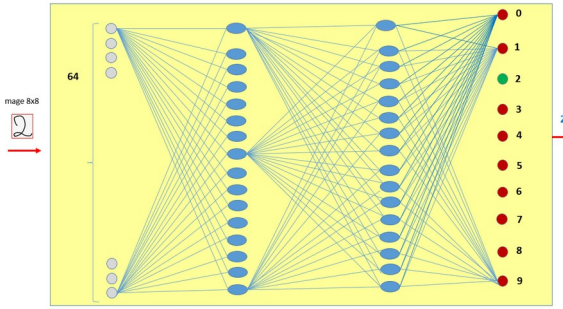


5

Complex classification problem



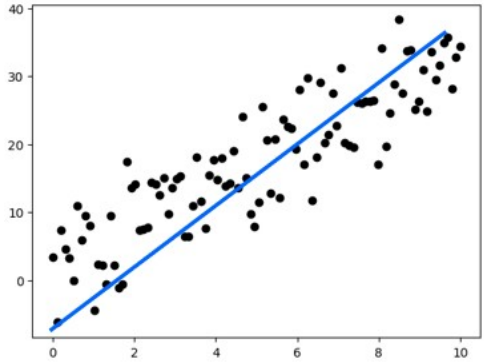
Hand Written digits



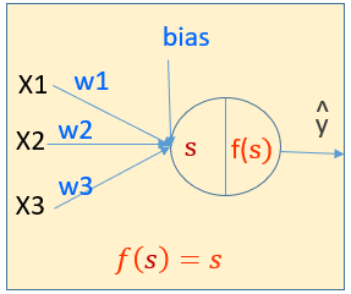
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

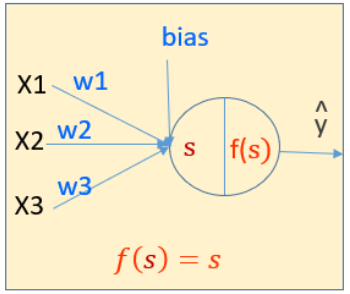


2

Multi Variate Linear Regression Problem

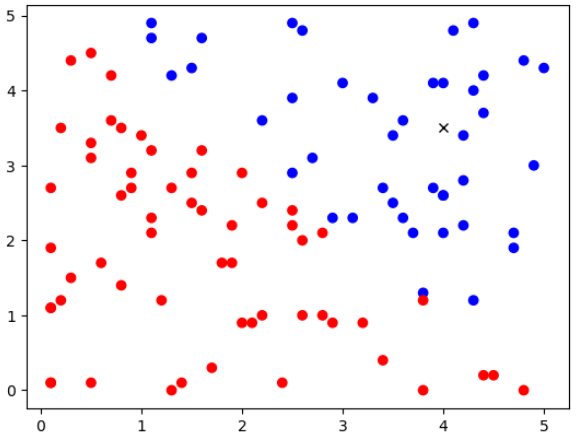
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising p

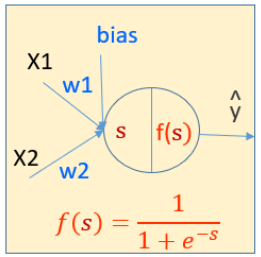


3

Linearly Separable problem

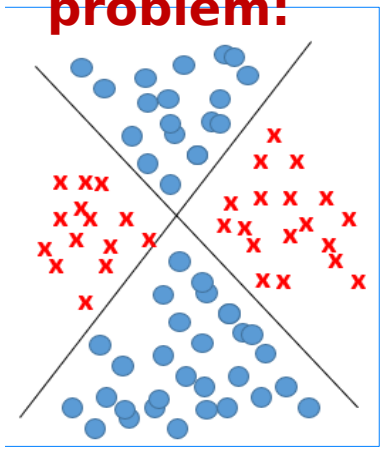


Red or Blue?

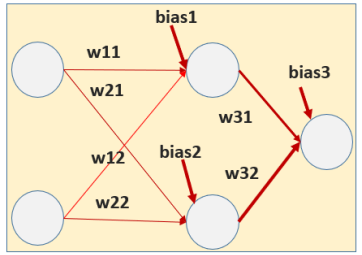


4

Non Linearly Separable problem:

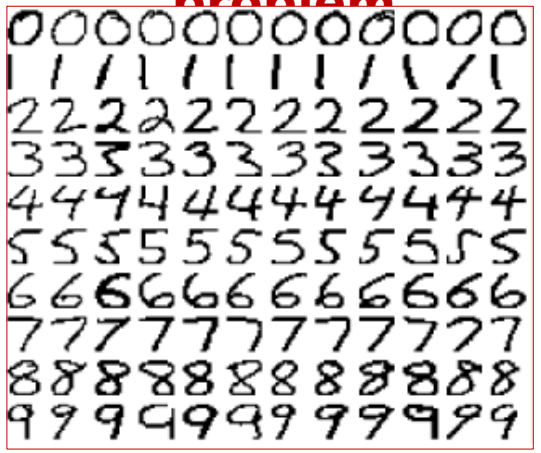


XOR Problem

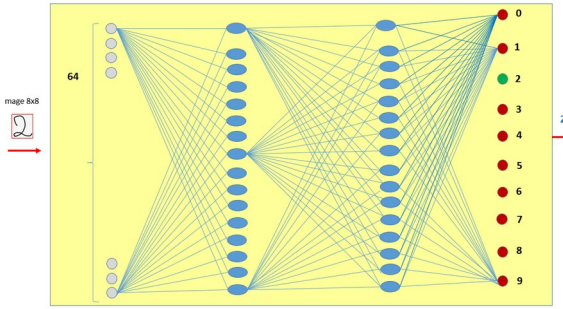


5

Complex classification problem



Hand Written digits



Génerer le dataset

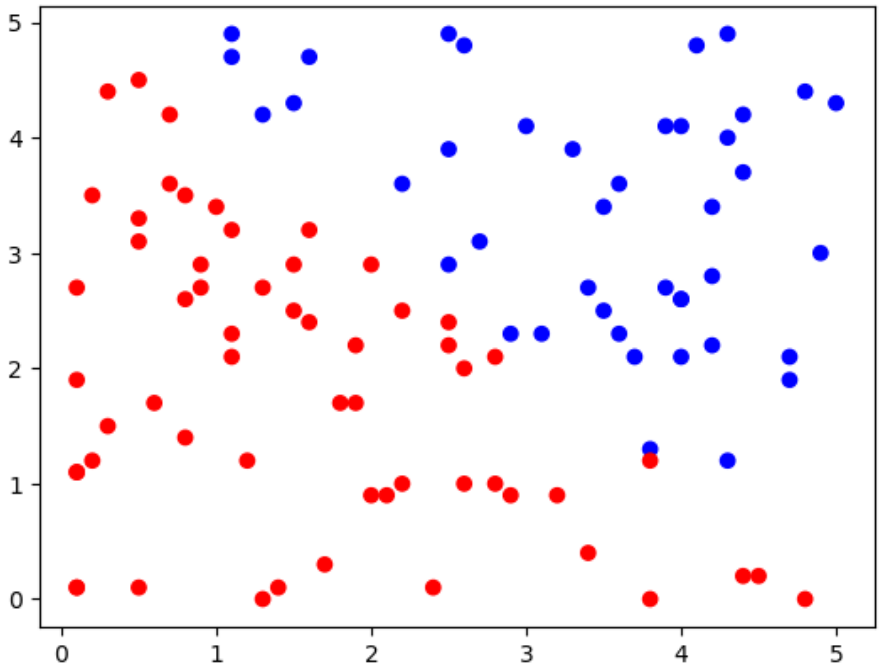
```
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.colors as clr
```

```
def f(x):
    return -x+5
```

```
# generate random floating point values
from random import seed
from random import random
# seed random number generator
def generateData(n):
    seed(1)
    min=0
    max=5
    # generate random numbers between
0-1
    L=[]
    for _ in range(n):
        valueX = random()
        scaledvalueX = min + (valueX *
(max - min))
        valueY = random()
        scaledvalueY = min + (valueY *
(max - min))
        x=round(scaledvalueX,1)
        y=round(scaledvalueY,1)
        y0=f(x)
        cl=0
        if(y>y0):
            cl=1
```

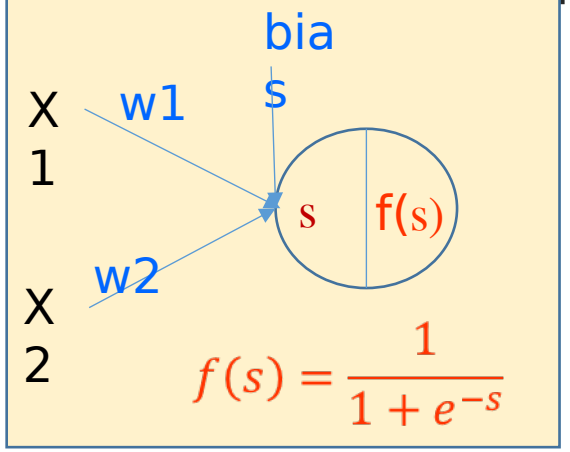
```
data=generateData
(100)
data=np.array(L)
X1=data[:,0]
X2=data[:,1]
```

```
colors = ['red','blue']
plt.scatter(X1,X2,c=C,cmap=clr.ListedColormap(colors))
```



Classification Problem by using neural network (1 neurone)

Af[illegible]lèle est:



$weighted\ sum = bias + w1 * X1 + \cdots + wm * Xm$

Activation: Identity or linear

```
def sigmoid(Z):
    return 1/(1+np.exp(-Z))
```

```
def predict(X):
    ws=bias+np.dot(X,W)
    yhat=sigmoid(ws)
    return yhat
```

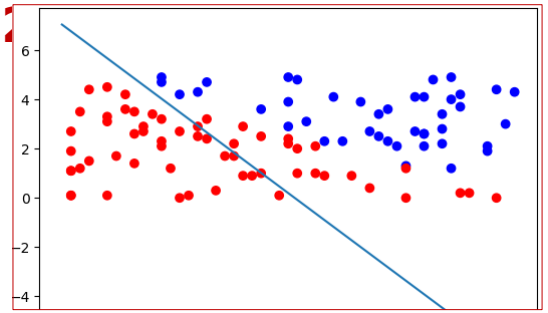
```
lr=0.01
def fit(X1,X2,y):
    n=len(y)
    w1=0.1
    w2=0.1
    bias=0.1
    for k in range(10000):
        ws=bias+w1*X1+
        yhat=sigmoid(ws)

        dbias=-(2/n)*np.sum(y-yhat)
        dw1=-(2/n)*np.sum(X1*(y-
yhat))
        dw2=-(2/n)*np.sum(X2*(y-
yhat))
        bias=bias-dbias*lr
        w1=w1-dw1*lr
        w2=w2-dw2*lr
    return bias,w1,w2
```

initialize the model

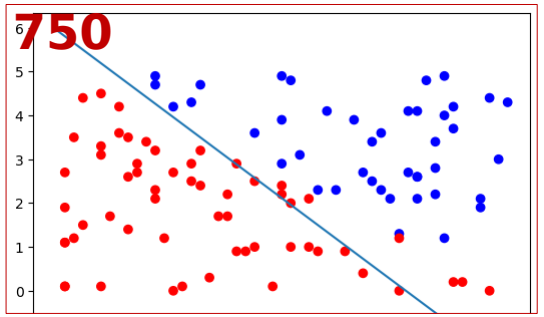
Predict using the model

Update the model

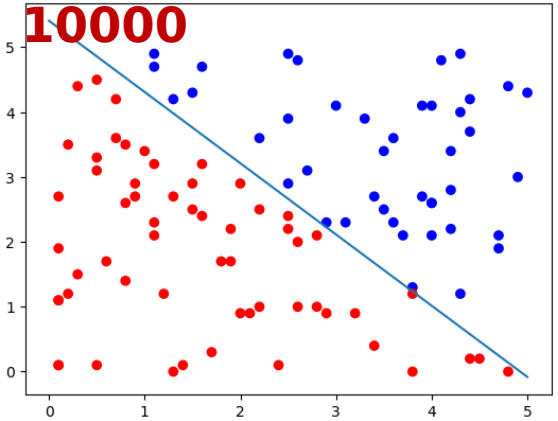
Iter 1



Iter 750

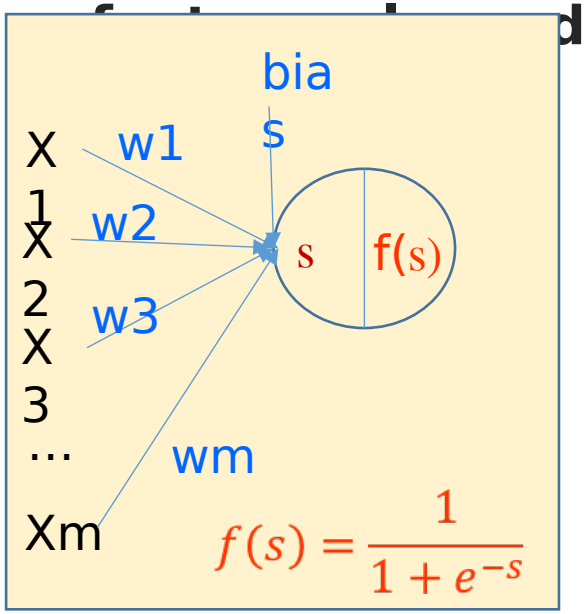


Iter 10000



Classification Problem by using neural network (1 neurone)

dèle est:



$$f(s) = \frac{1}{1+e^{-s}}$$

$$weighted\ sum = bias + w1 * X1 + \cdots + wm * Xm$$

Activation: Identity or linear

```
def sigmoid(Z):
    return 1/(1+np.exp(-Z))
```

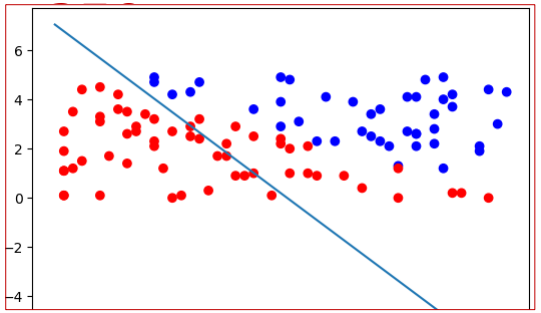
```
lr=0.01
def fit(X,y):
    n=len(y)

W=np.array(np.zeros(X.shape[1]))
)
    bias=0
    for k in range(10000):
        ws=bias+np.do
        yhat=sigmoid(ws)

        dbias=-(2/n)*np.sum(y-yhat)
        dW=-(2/n)* np.dot(X.T,(y-yhat))
        bias=bias-dbias*lr
        W=W-dW*lr

    return bias,W
```

initialize the model

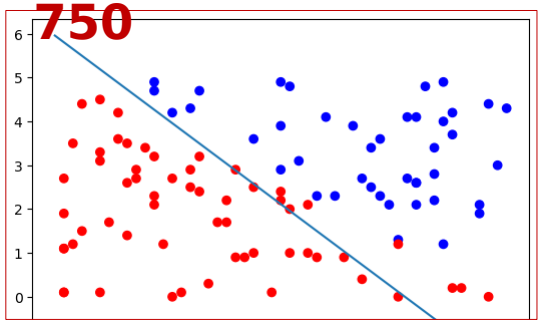
Predict using the model

Update the model

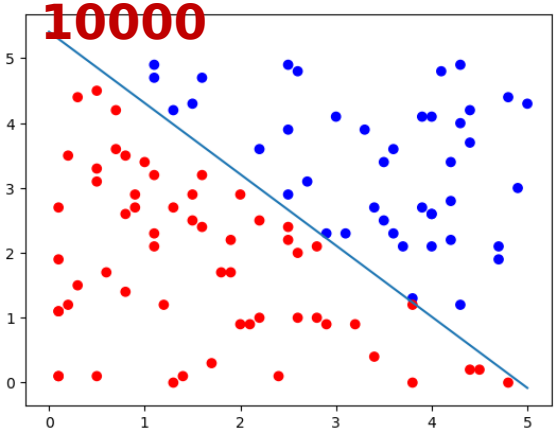
Iter



Iter 750



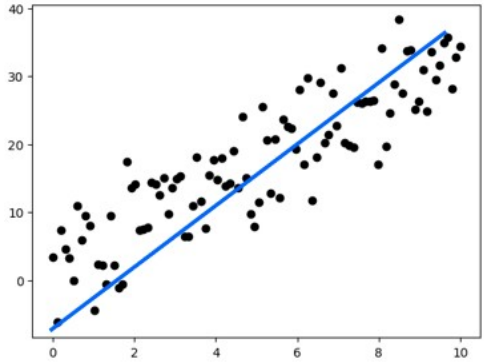
Iter 10000



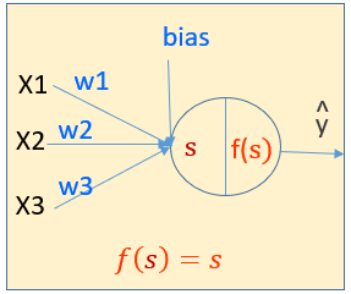
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

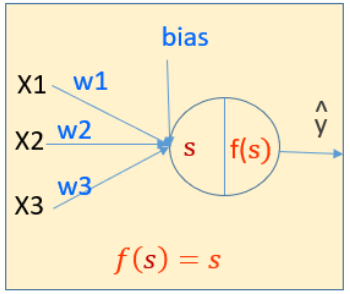


2

Multi Variate Linear Regression Problem

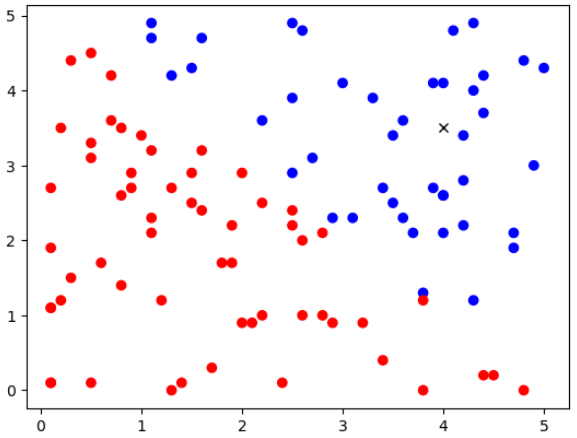
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
…			
199.8	2.6	21.2	**10,6**

Advertising p

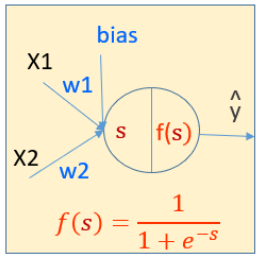


3

Linearly Separable problem

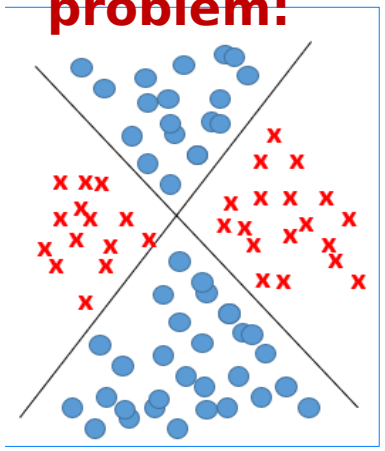


Red or Blue?

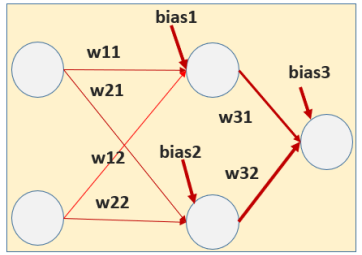


4

Non Linearly Separable problem:

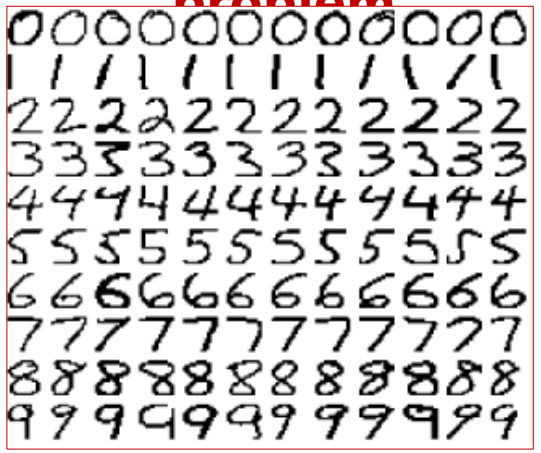


XOR Problem

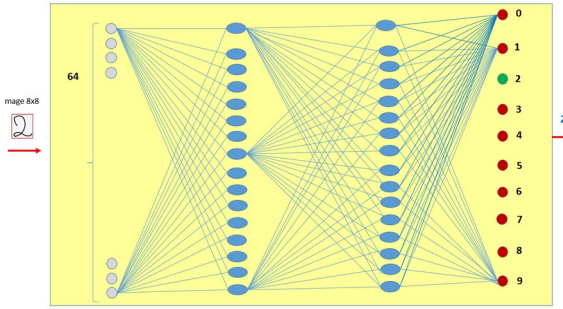


5

Complex classification problem



Hand Written digits

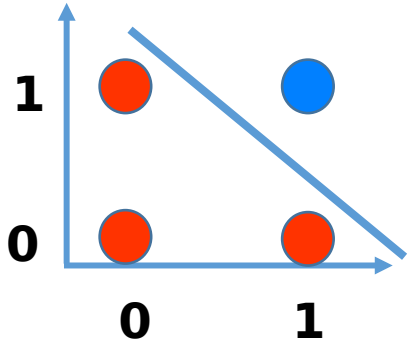


Problem

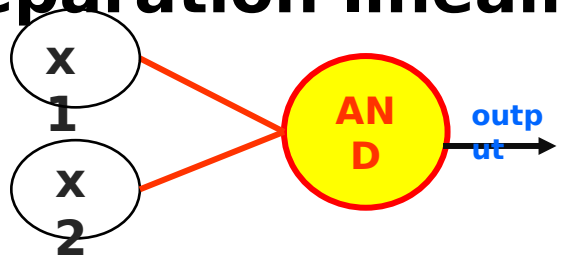
Séparation de
AND, OR,NAND, NOR, XOR

And

	F	T
T	F	T
F	F	F

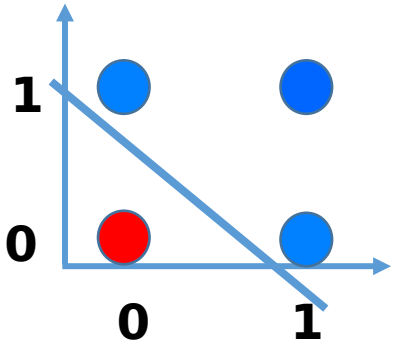


On peut avoir une séparation linéaire

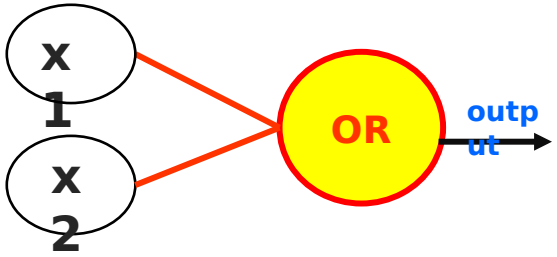


OR

	F	T
T	T	T
F	F	T

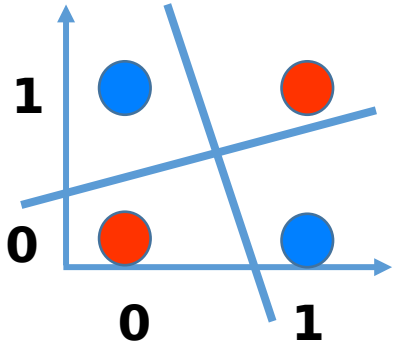


On peut avoir une séparation linéaire



XOR

	F	T
T	T	F
F	F	T

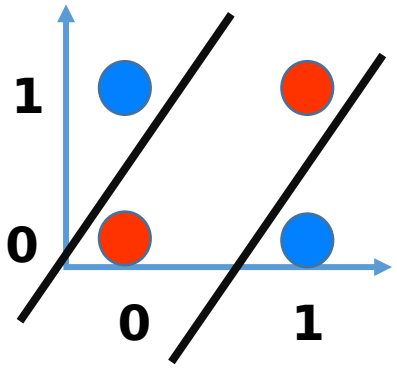


On ne peut pas avoir une séparation linéaire

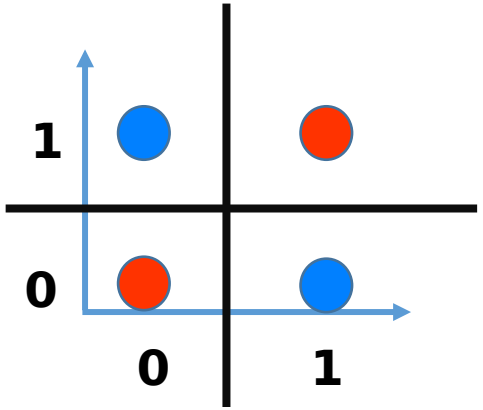
Séparation de X0R

XOR

	F	T
T	T	F
F	F	T

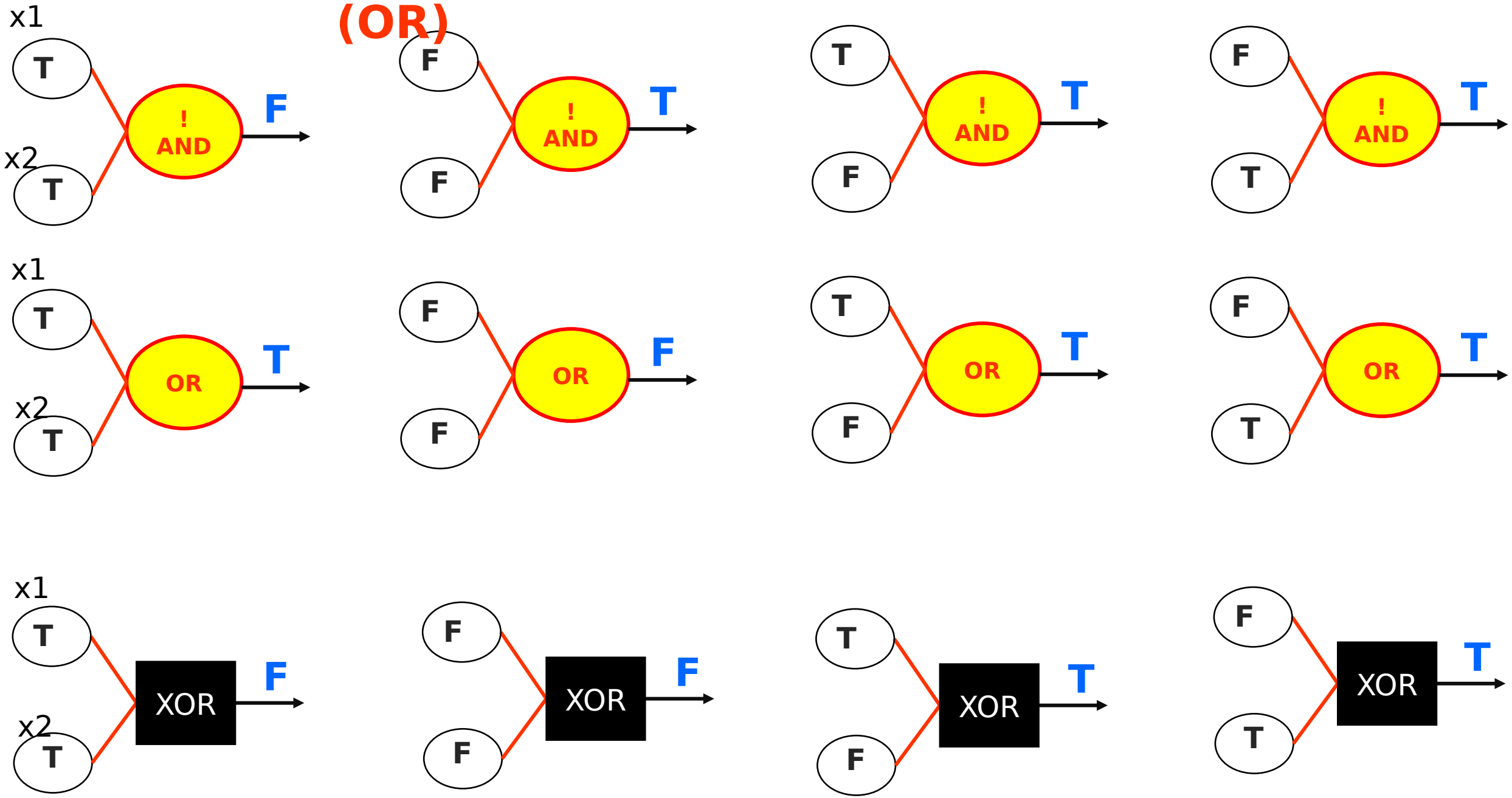


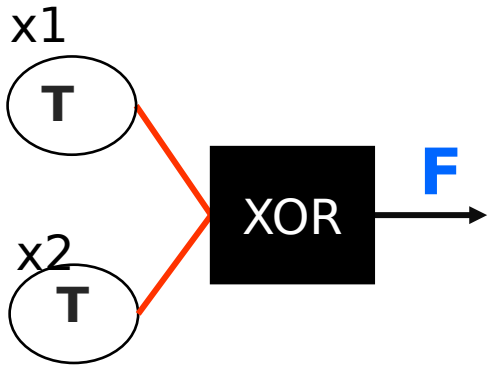
On peut utiliser plus qu'une ligne pour faire la séparation

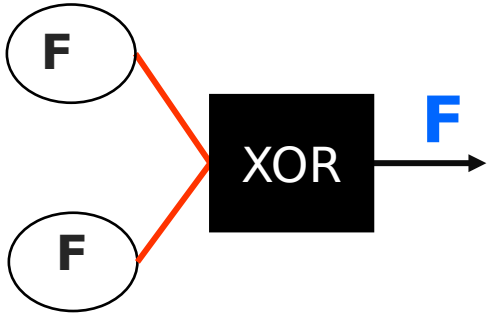


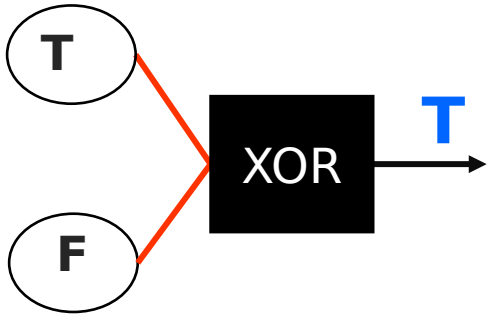
XOR = (!AND) and (OR)

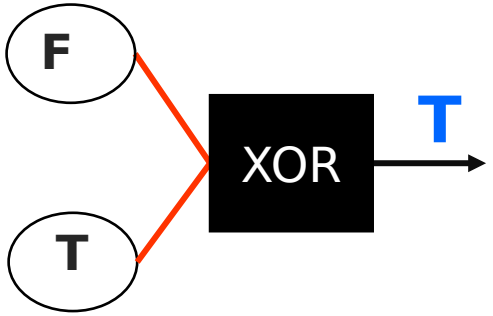
XOR = (!AND) and (OR)

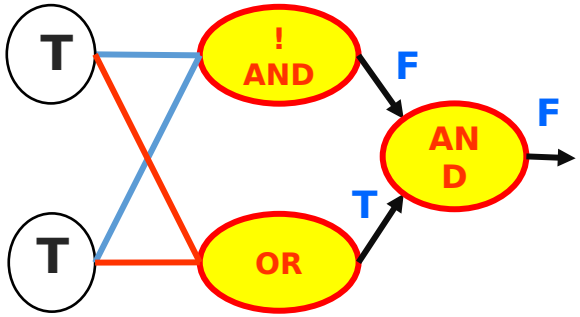


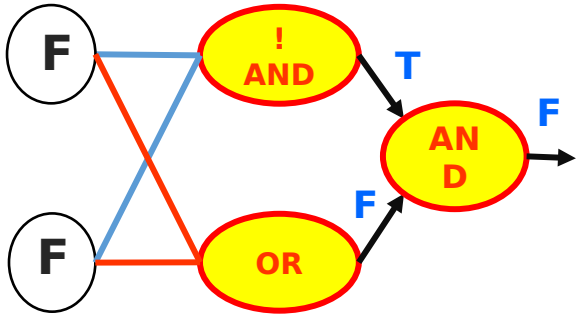

x1
T
x2
T
XOR
F

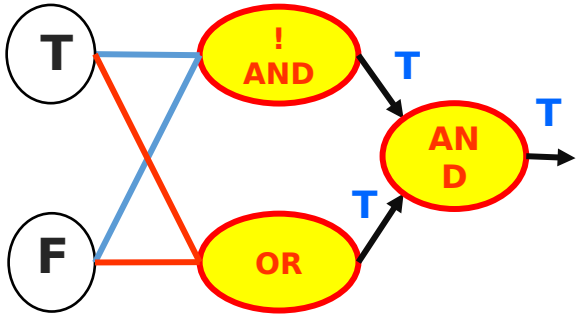

F
F
XOR
F

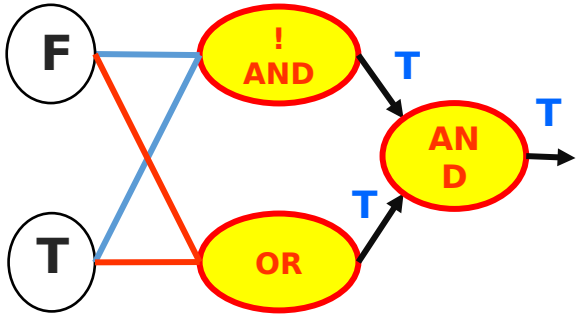

T
F
XOR
T

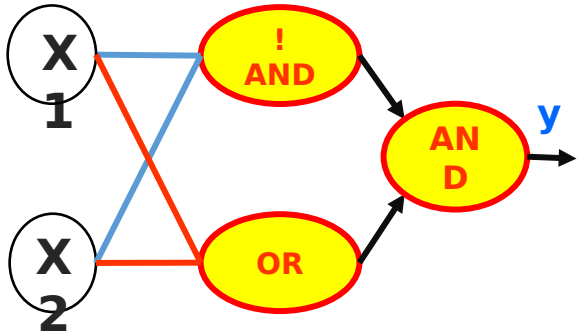

F
T
XOR
T


T
T
! AND
OR
F
T
AN D
F


F
F
! AND
OR
T
F
AN D
F


T
F
! AND
OR
T
T
AN D
T

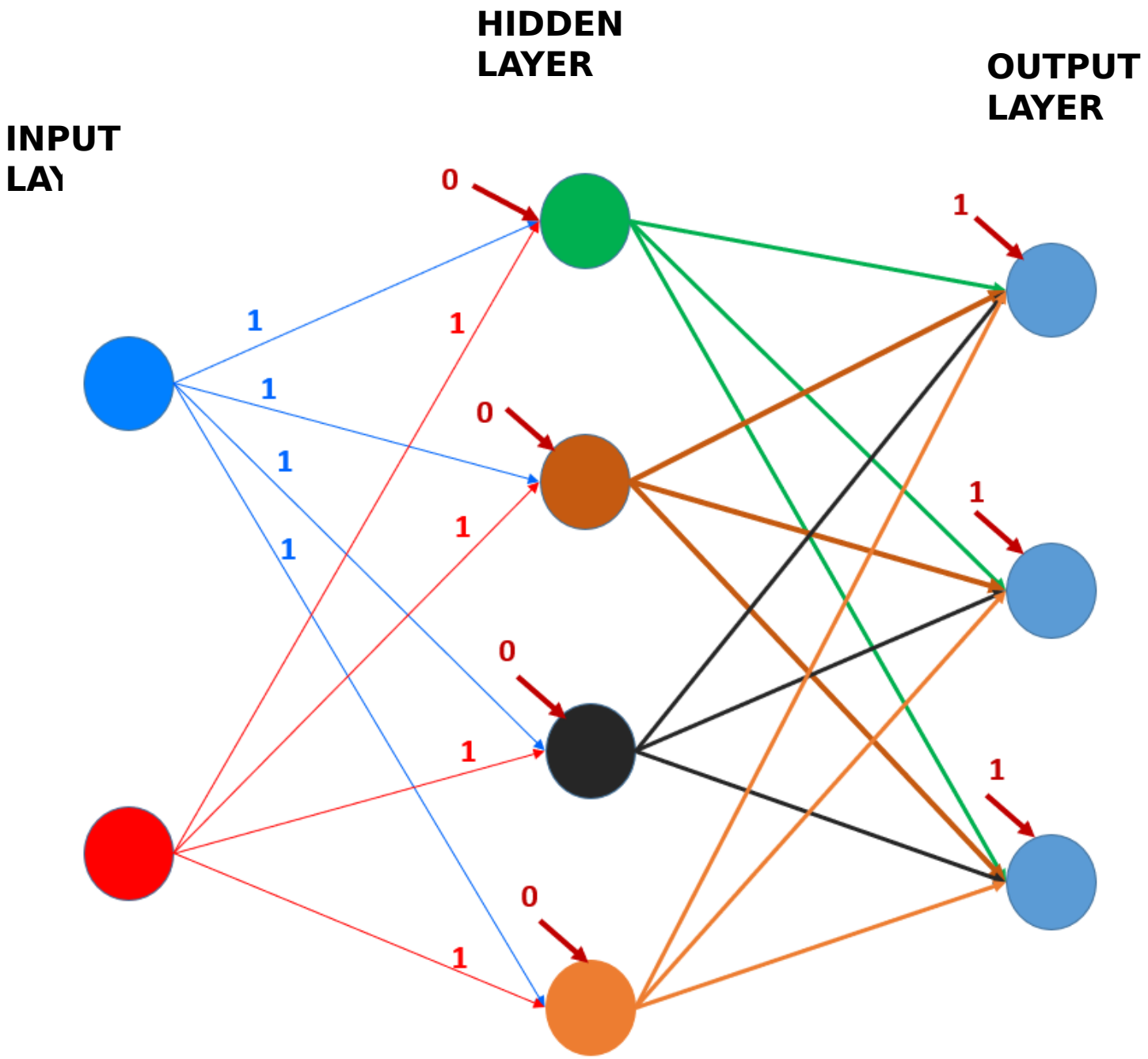

F
T
! AND
OR
T
T
AN D
T


X 1
X 2
! AND
OR
AN D
y

Utiliser keras pour créer un réseau de neurones avec plusieurs neurones (couche, layer)

Ce qu’il faut connaitre:

1. **Comment créer un multilayer neural network (en général)?**
2. Découvrir les éléments du neural network créé: W, bias


HIDDEN LAYER
OUTPUT LAYER
INPUT LAY
0
1
0
1
0
1
1
1
1
1
1
1
1
1

Modèle et ses paramètres

1. Créer le modèle
2. Créer une couche et l’ajouter au modèle
3. Paramètres d’une couche

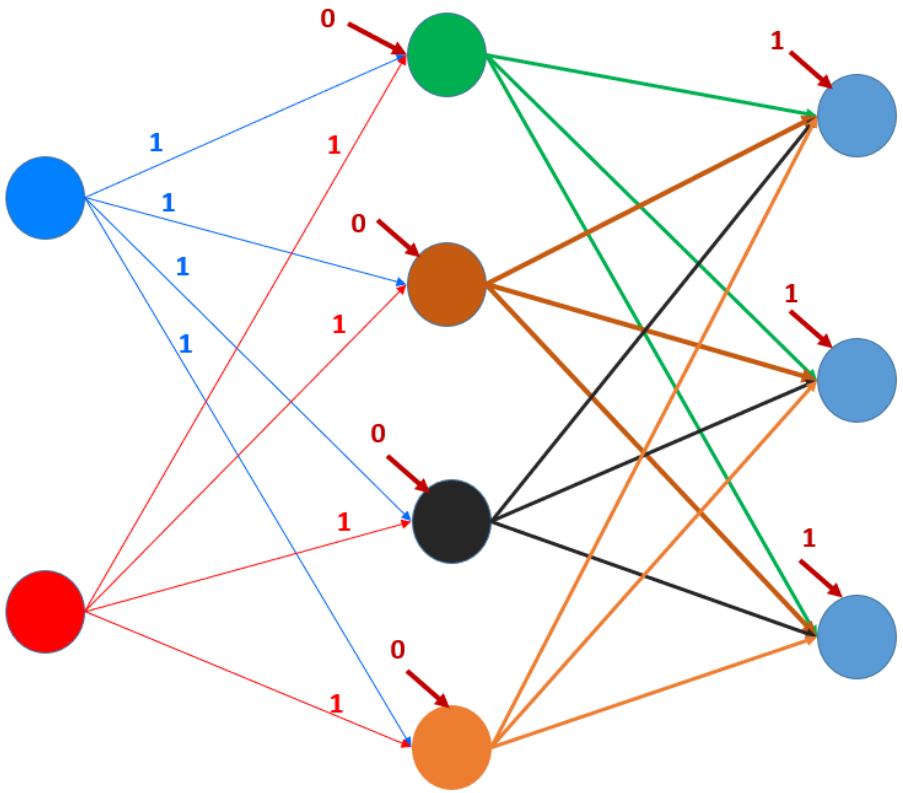
Model et ses paramètres

```
from keras.models import Sequential
from keras.layers import Flatten, Dense, Activation
```

```
model = Sequential()
```

```
#input_layer=Flatten(input_shape=[2, 1])
#model.add(input_layer)
hidden_layer=Dense(4,
input_dim=2,kernel_initializer='ones',bias_initializer='zeros
')
model.add(hidden_layer)
output_layer=Dense(3,
kernel_initializer='random_uniform',bias_initializer='ones')
model.add(output_layer)
print(model.summary())
```

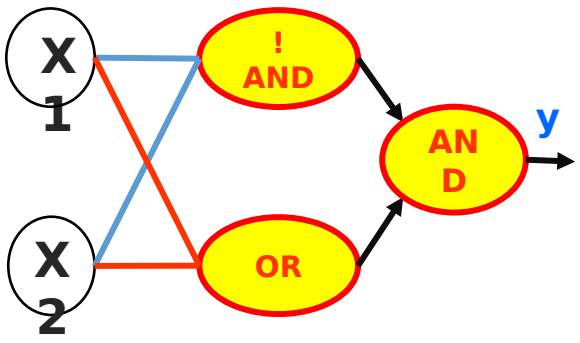
```
model.compile(loss="categorial_crossentropy",
            optimizer="adam",
            metrics=["accuracy"])
```



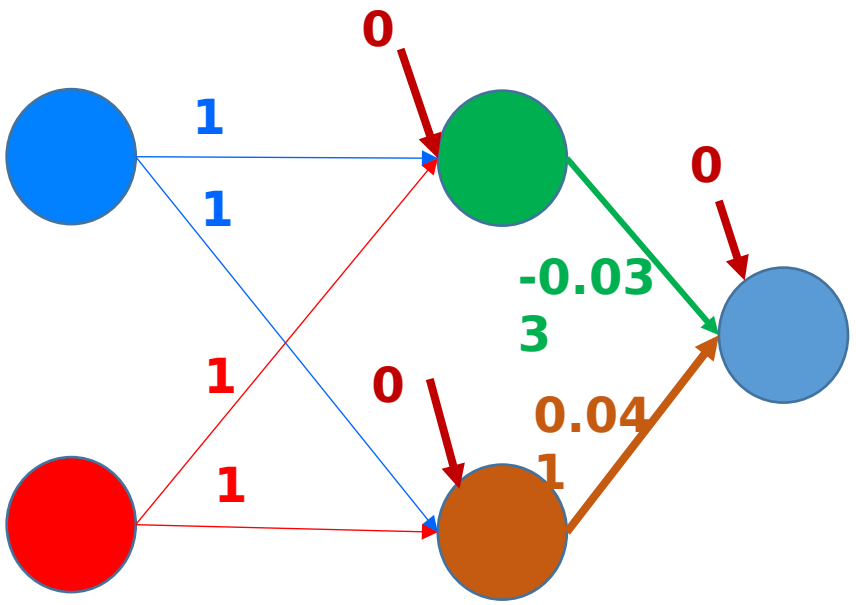

```
[array([[1., 1., 1., 1.],[1., 1., 1., 1.]],
dtype=float32),
       array([0., 0., 0., 0.], dtype=float32),
 array([[-0.03575971, -0.02119735,
0.01558626],
     [ 0.04222678, -0.04578313,  0.0360521 ],
     [-0.02753322,  0.00745941,
-0.04211374],
     [-0.04299896, -0.04190966,
0.02975568]], dtype=float32),
     array([1., 1., 1.], dtype=float32)]
```

Revenons à XOR Problem

Les étapes à suivre pour créer le modèle



1. **Data**
2. **Model**
   - Create the model, compile the model: keras, tensorflow
     - Optimizer function: adam
     - Loss function: binary_crossentropy
   - Train the model
   - Parameters
3. **Prediction**
4. **Evaluation**

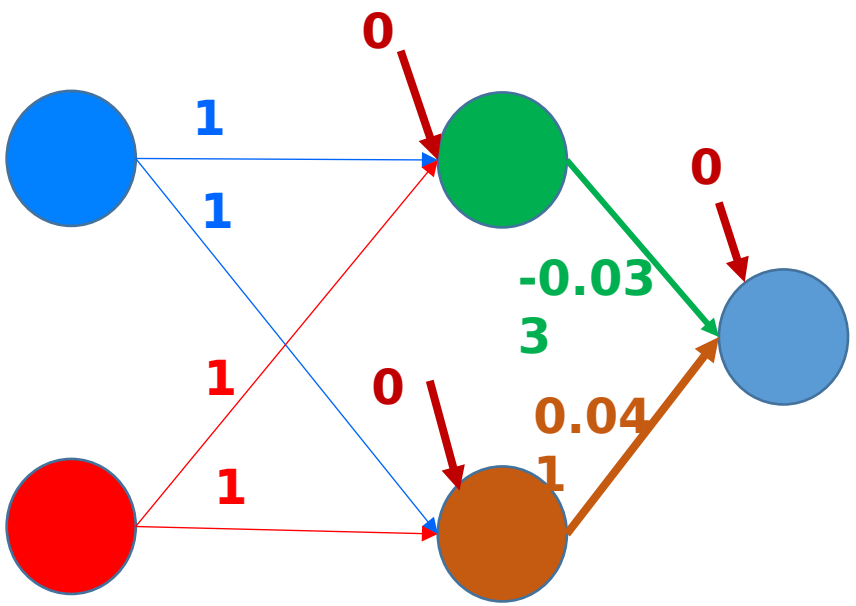


2. Model

```
from keras.models import Sequential

from keras.layers import Activation, Dense,Flatten

model = Sequential()

#input_layer = Flatten(input_shape=[2,1])

hidden_layer = Dense(2,input_dim=2,
kernel_initializer='ones',bias_initializer='zeros', activation='relu')

output_layer =
Dense(1,kernel_initializer='random_uniform',bias_initializer='zero
s', activation='sigmoid')

#model.add(input_layer)

model.add(hidden_layer)

model.add(output_layer)

coef=model.get_weights()

print(coef)
```

```
model.compile(loss='binary_crossentropy',optimizer='adam',metri
cs=['accuracy'])
```

```
model.fit(X,y,epochs=1000)

print(model.get_weights())

print(model.predict(X)
```



[array([[1., 1.], [1., 1.]], dtype=float32), array([0., 0.], dtype=float32), array([[-0.03330588], [0.04190588]], dtype=float32), array([0.], dtype=float32)]

2. Model (avec une autre fonction d'activation)

```
from keras.layers import LeakyReLU
act = LeakyReLU(alpha = 0.3)
```

```
model = Sequential()

hidden_layer = Dense(2,input_dim=2, kernel_initializer='ones',bias_initializer='zeros',activation=act)

model.add(hidden_layer)

output_layer = Dense(1,kernel_initializer='random_uniform',bias_initializer='zeros',activation='sigmoid')

model.add(output_layer)

model.add(hidden_layer)

model.add(output_layer)

coef=model.get_weights()

print(coef)
```

```
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
```

```
model.fit(X,y,epochs=1000,batch_size=4)

print(model.get_weights())

print(model.predict(X))
```

Comment mettre à jour les poids dans un réseau avec des couches cachées

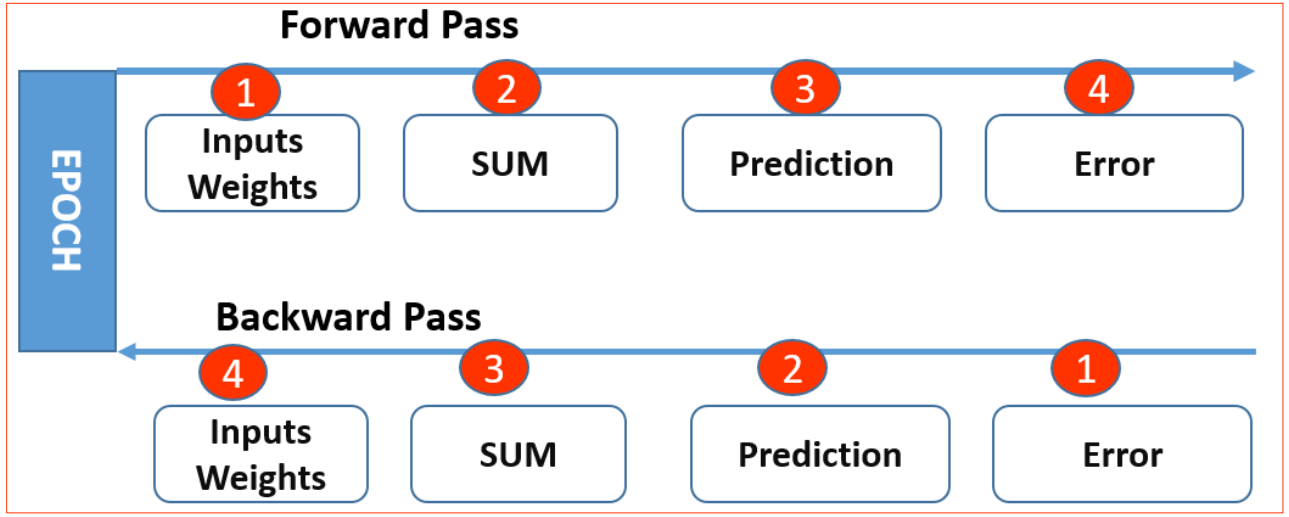
- Back propagation
- Chaine rule

Dans la suite, nous allons appliquer le chain rule:

- pour un seul neurone (au lieu de mettre à jour les poids directement)
- Pour un réseau avec une couche cachée

Backpropagation Algorithm

- Après chaque passage en avant (**Forward pass**) à travers un réseau, la rétropropagation effectue un passage en arrière (**Backward pass**) tout en ajustant les paramètres du modèle (poids et biais).
- L'algorithme est utilisé pour entraîner efficacement un réseau de neurones à l'aide d'une méthode appelée **chain rule**



chain rule

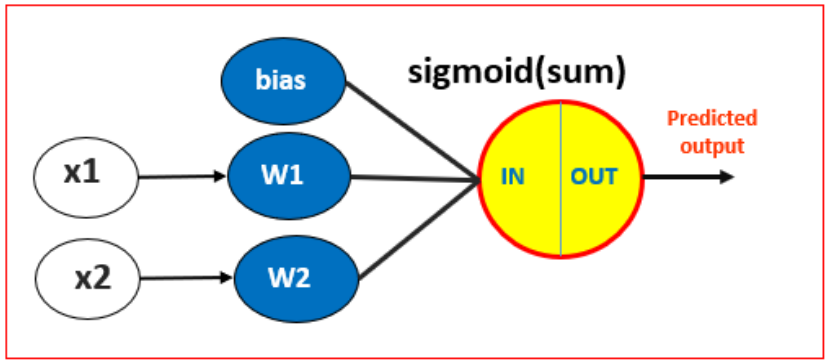
$y = f(u)$

et $u = g(x)$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

1. Forward pass: pour la prédiction
2. Backward pass: pour la correction

Algorithme



De 1 à n_epochs faire

- Forward pass
- Backward pass

1 **Initialize weights** **Initialize learning rate**

De 1 à n_epochs faire

2

$$sum = \sum_{i=0}^{n} W_i * X_i$$

3 **Activation function →**

$$f(s) = \frac{1}{1+e^{-s}} = output$$

4 **error= desired-**

$$Loss = \frac{1}{2}(desired - predicted)^2$$

5 **Weights update**

$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

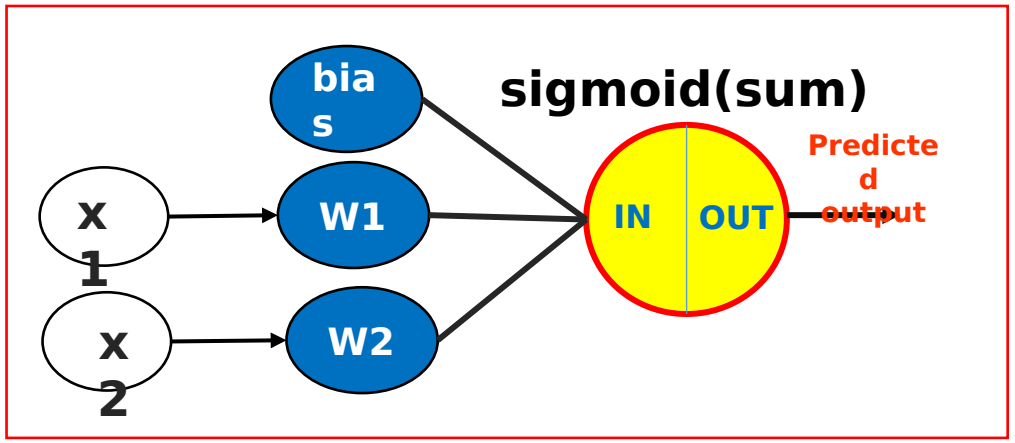
$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

$$bias = bias - learning_rate * \frac{\partial loss}{\partial bias}$$

Exemple

X1	X2	output
0.1	0.3	0.03
…		

learning_rate=0.01



- Initialiser les poids de manière aléatoire
- Mettre à jour les poids en appliquant l’algorithme de backpropabation pour minimiser la fonction Loss
- Tourner l’algorithme en deux itérations

Appliquer le chain rule pour un seul neurone

X1	X2	output
0.1	0.3	0.03
…		

learning_rate= 0.01

Forward pass

Sum=w1*x1+w2*x2+bias

Predicted output=sigmoid(sum)

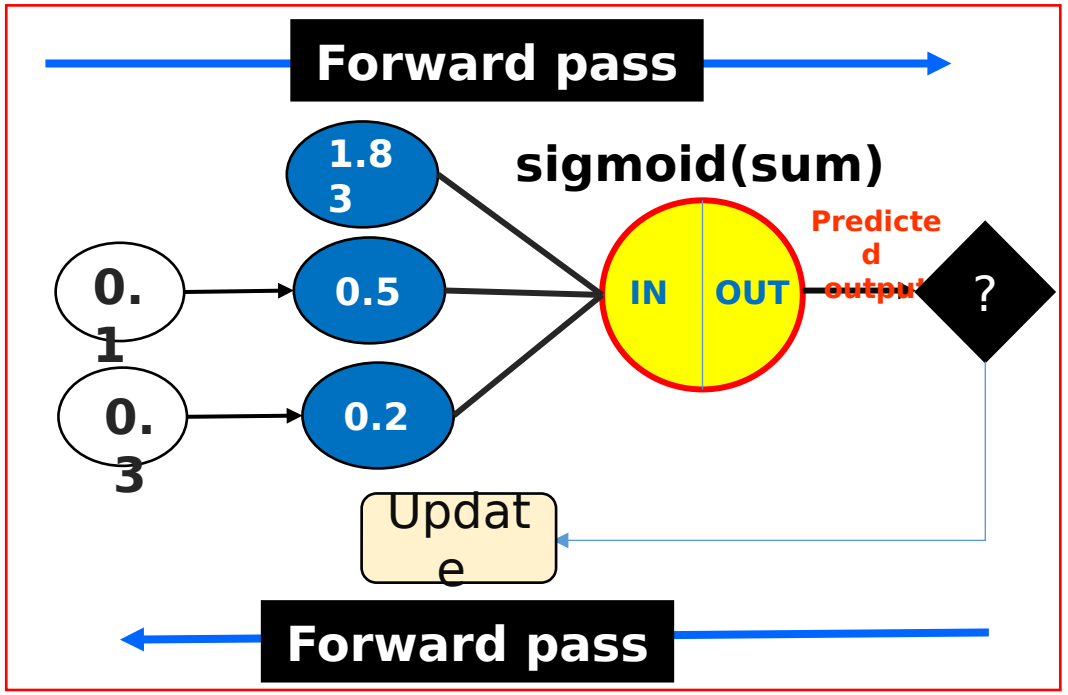
$$Loss = \frac{1}{2}(desired - predicted)^2$$

Backward pass

$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

$$bias = bias - learning_rate * \frac{\partial loss}{\partial bias}$$



$$\frac{\partial Loss}{\partial w1} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w1}$$

$$\frac{\partial Loss}{\partial w2} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w2}$$

$$\frac{\partial Loss}{\partial bias} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial bias}$$

$$\frac{\partial Loss}{\partial predicted} = predicted - desired$$

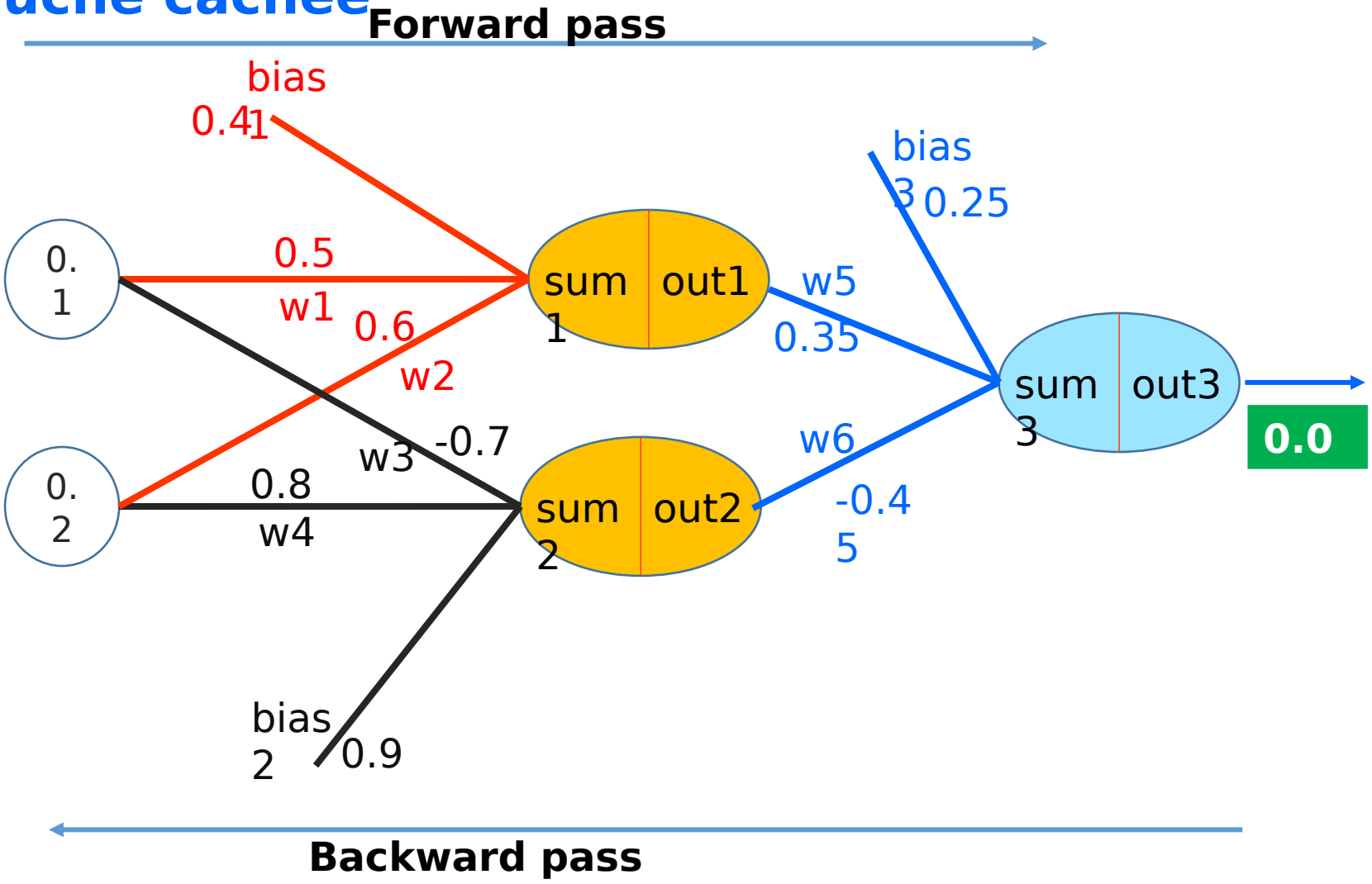
$$\frac{\partial predicted}{\partial s} = f(s)\,(1 - f(s))$$

$$\frac{\partial s}{\partial w1} = x1$$

$$\frac{\partial s}{\partial w2} = x2$$

$$\frac{\partial s}{\partial bias} = 1$$

Appliquer le chain rule pour un réseau de neurones avec une couche cachée



Comment?

$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

$$w_3 = w_3 - learning_rate * \frac{\partial loss}{\partial w_3}$$

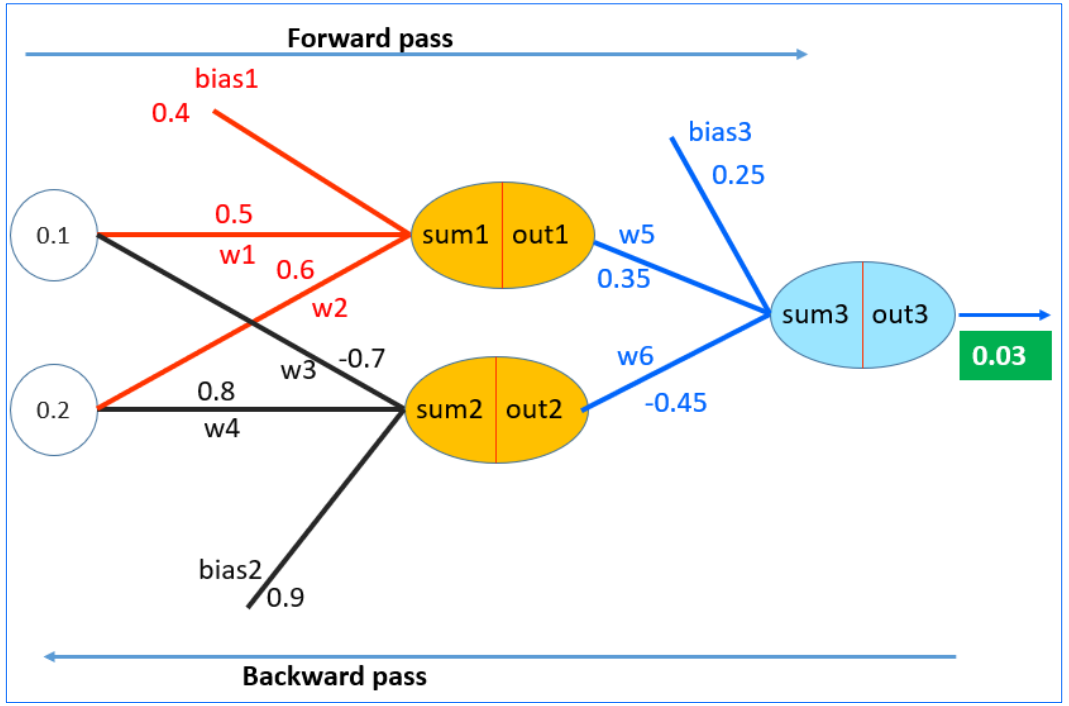
$$w_4 = w_4 - learning_rate * \frac{\partial loss}{\partial w_4}$$

$$w_{hidden} = w_{hidden} - learning_rate * \frac{\partial loss}{\partial w_{hidden}}$$

$$bias_1 = bias_1 - learning_rate * \frac{\partial loss}{\partial bias_1}$$

$$bias_2 = bias_2 - learning_rate * \frac{\partial loss}{\partial bias_2}$$

$$bias_{hidden} = bias_{hidden} - learning_rate * \frac{\partial loss}{\partial bias_{hidden}}$$



$$w_5 = w_5 - learning_rate * \frac{\partial loss}{\partial w_5}$$

$$w_6 = w_6 - learning_rate * \frac{\partial loss}{\partial w_6}$$

$$w_{out} = w_{out} - learning_rate * \frac{\partial loss}{\partial w_{out}}$$

$$bias_3 = bias_3 - learning_rate * \frac{\partial loss}{\partial bias_3}$$

$$bias_{out} = bias_{out} - learning_rate * \frac{\partial loss}{\partial w_{bias}}$$

1) Update hidden layer weights and bias

Update hidden layer weights and bias

$$w_{hidden} = w_{hidden} - learning_rate * \frac{\partial loss}{\partial w_{hidden}}$$

Calculer $\frac{\partial Loss}{\partial W_hidden}$

$$bias_{hidden} = bias_{hidden} - learning_rate * \frac{\partial loss}{\partial bias_{hidden}}$$

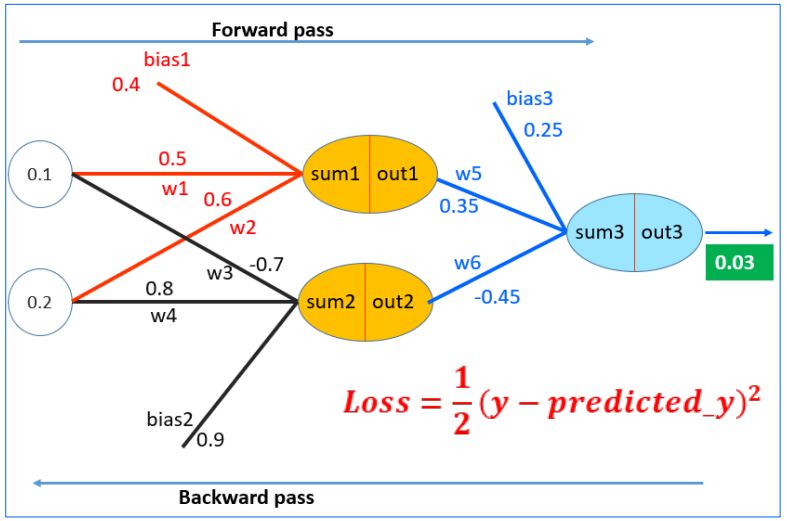
Calculer $\frac{\partial Loss}{\partial bias_hidden}$

Update hidden layer weights

$$\frac{\partial Los}{\partial W_hid}$$

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial W_hidden}$



$$\frac{\partial Loss}{\partial w1} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out1} * \frac{\partial out1}{\partial sum1} * \frac{\partial sum1}{\partial w1}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout1= w5

out1 =sigmoid(sum1)

d_out1_dsum1=sigmoid_derivative(out1)

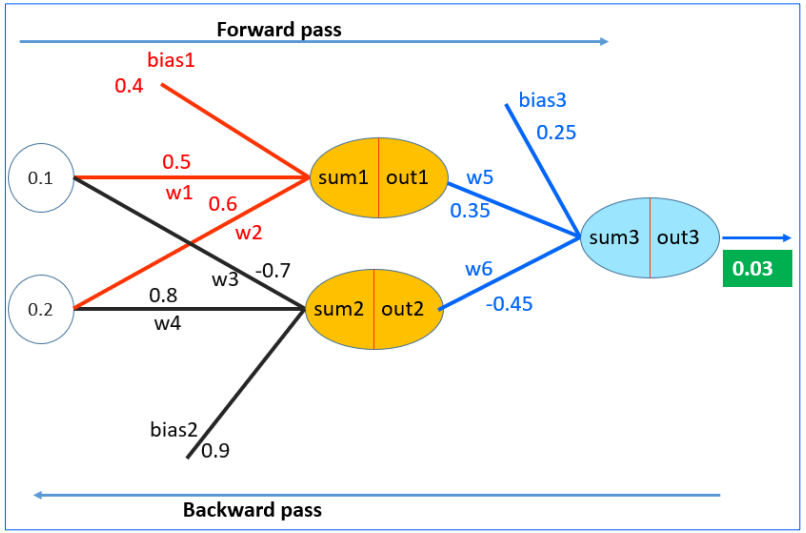
sum1 =w1*x1+w2*x2+bias1

d_sum1_dw1 =x1

dloss_dw1=(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* x1

Backpropagation – MultiLayer NN

$$Loss = \frac{1}{2}(y - predicted_y)^2$$



$$\frac{\partial Loss}{\partial w2} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out1} * \frac{\partial out1}{\partial sum1} * \frac{\partial sum1}{\partial w2}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout1= w5

out1 =sigmoid(sum1)

d_out1_dsum1=sigmoid_derivative(out1)

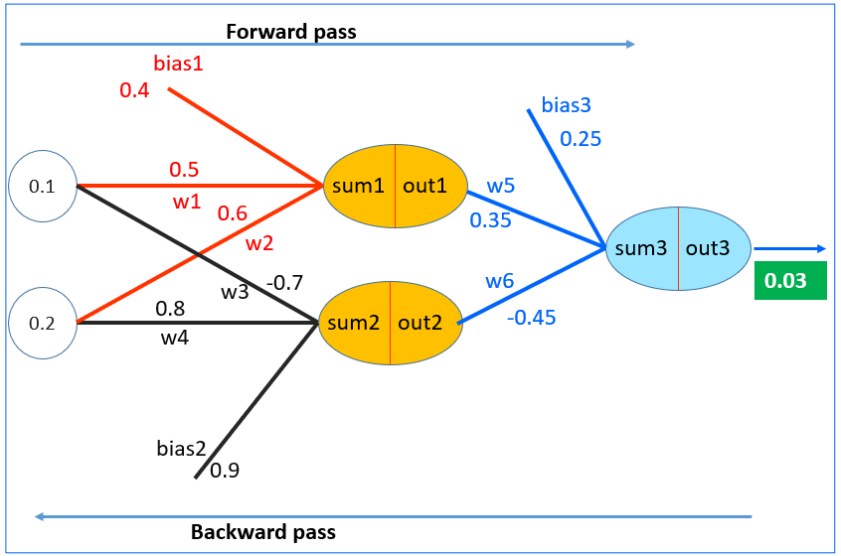
sum1 =w1*x1+w2*x2+bias1

d_sum1_dw2 =x2

dloss_dw2=**(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* x2**

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial W_hidden}$



$$\frac{\partial Loss}{\partial w3} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out2} * \frac{\partial out2}{\partial sum2} * \frac{\partial sum2}{\partial w3}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout2= w6

out2 =sigmoid(sum2)

d_out2_dsum2=sigmoid_derivative(out2)

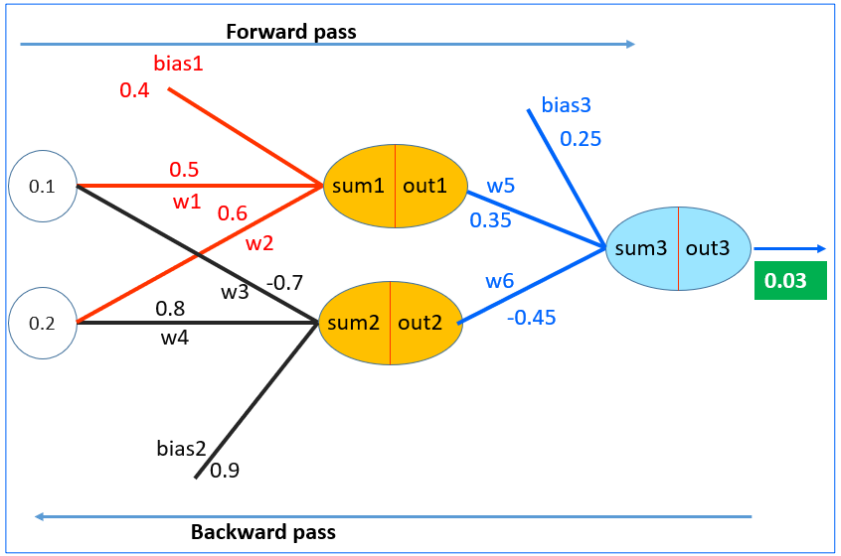
sum2 =w3*x1+w4*x2+bias2

d_sum2_dw3 =x1

dloss_dw3=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* x1

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial W_hidden}$



$$\frac{\partial Loss}{\partial w4} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out2} * \frac{\partial out2}{\partial sum2} * \frac{\partial sum2}{\partial w4}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout2= w6

out2 =sigmoid(sum2)

d_out2_dsum2=sigmoid_derivative(out2)

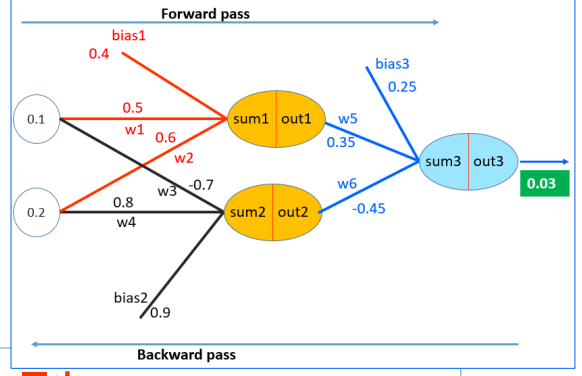
sum2 =w3*x1+w4*x2+bias2

d_sum2_dw4 =x2

dloss_dw4=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* x2

Update hidden layer weights

$$\frac{\partial Loss}{\partial W_hidden}$$



dloss_dw1=(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* x1

dloss_dw2=(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* x2

dloss_dw3=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* x1

dloss_dw4=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* x2

dloss_dw=(predicted_y-y)* sigmoid_derivative(predicted_y)* output_W* sigmoid_derivative(h_out)* X

Code python

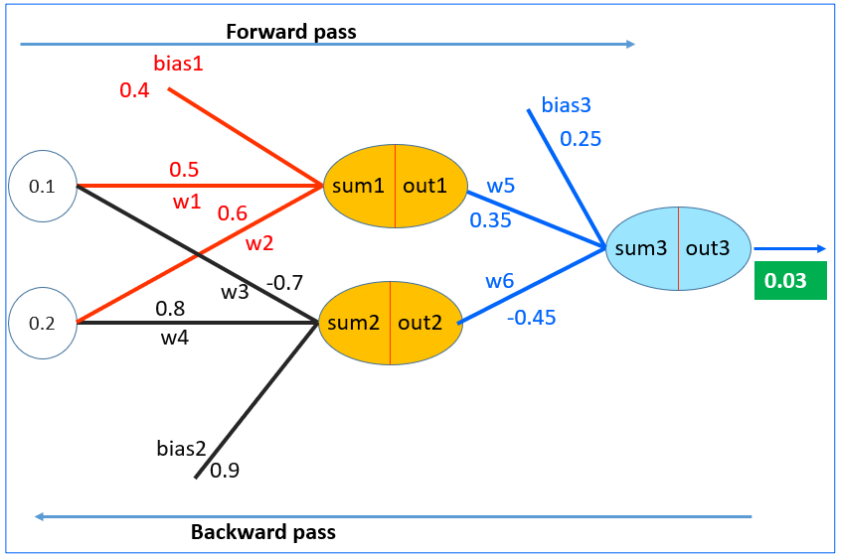
```
dloss_dw_h= X.T.dot((predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out))
```

Update hidden layer bias

$$\frac{\partial Lo}{\partial bias_h}$$

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial bias_hidden}$



$$\frac{\partial Loss}{\partial bias1} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out1} * \frac{\partial out1}{\partial sum1} * \frac{\partial sum1}{\partial bias1}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout1= w5

out1 =sigmoid(sum1)

d_out1_dsum1=sigmoid_derivative(out1)

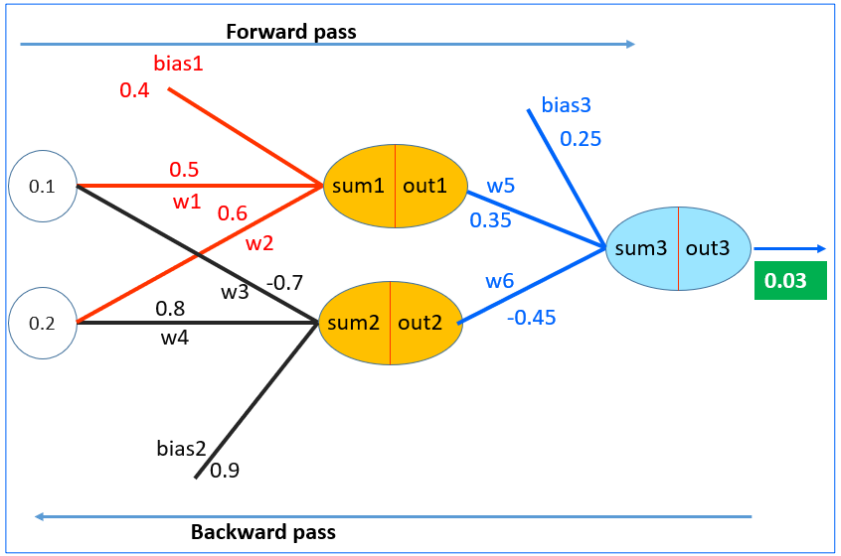
sum1 =w1*x1+w2*x2+bias1

d_sum1_bias1 =1

dloss_bias1=(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* 1

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial bias_hidden}$



$$\frac{\partial Loss}{\partial bias2} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out2} * \frac{\partial out2}{\partial sum2} * \frac{\partial sum2}{\partial bias2}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout2= w6

out2 =sigmoid(sum2)

d_out1_dsum2=sigmoid_derivative(out2)

sum2 =w3*x1+w4*x2+bias2

d_sum2_bias2 =1

dloss_bias2=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* 1

Update hidden layer bias

$$\frac{\partial Loss}{\partial bias_hidden}$$

```
dloss_bias1=(predicted_y-y)* sigmoid_derivative(predicted_y)* w5*
sigmoid_derivative(out1)* 1
dloss_bias2=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6*
sigmoid_derivative(out2)* 1
```

```
dloss_bias_h=(predicted_y-y)* sigmoid_derivative(predicted_y)* output_W *
sigmoid_derivative(h_out)
```

Code python

```
dloss_dbias_h=(predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out)
```

2) Update output layer weights and bias

Update output layer weights and bias

$$\frac{\partial Loss}{\partial Wout}$$

Calculer $w_{out} = w_{out} - learning_rate * \frac{\partial loss}{\partial w_{out}}$

$$\frac{\partial Loss}{\partial bias_out}$$

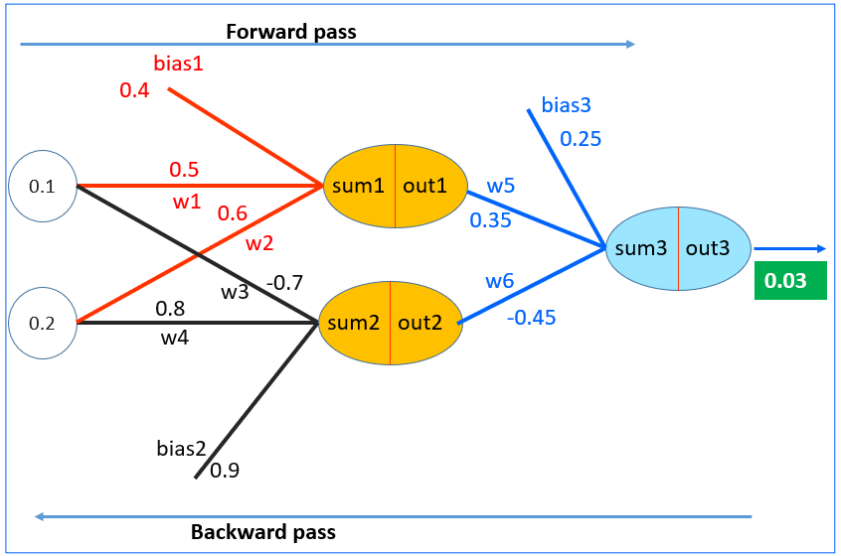
Calculer $bias_{out} = bias_{out} - learning_rate * \frac{\partial loss}{\partial w_{bias}}$

Update output layer weights

$$\frac{\partial Loss}{\partial Wout}$$

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial Wout}$



$$\frac{\partial Loss}{\partial w5} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial W5}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

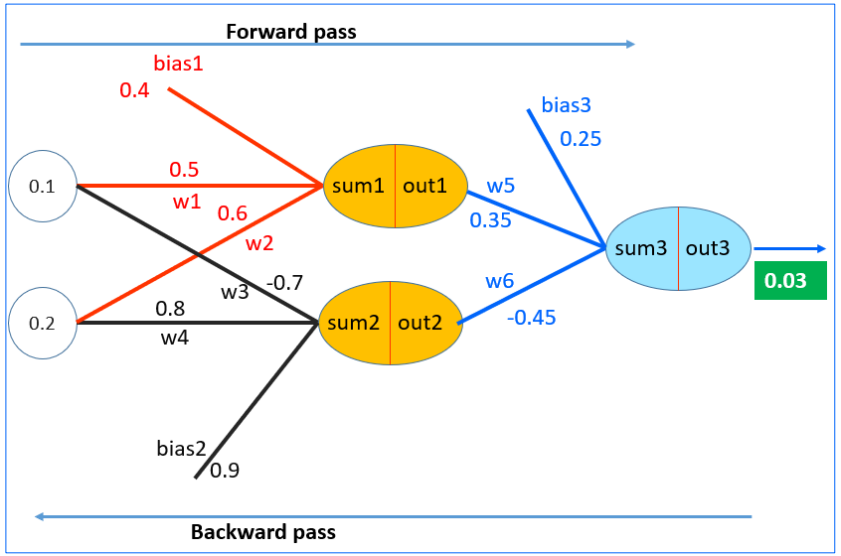
d_sum3_dw5 =out1

dloss_dw5=**(predicted_y-y)* sigmoid_derivative(predicted_y) * out1**

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial Wout}$

$$\frac{\partial Loss}{\partial w6} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial W6}$$



$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

d_sum3_dw6 =out2

dloss_dw6=(predicted_y-y)* sigmoid_derivative(predicted_y) * out2

$$\frac{\partial Loss}{\partial Wout}$$

```
dloss_dw5=(predicted_y-y)* sigmoid_derivative(predicted_y) * out1
```

```
dloss_dw6=(predicted_y-y)* sigmoid_derivative(predicted_y) * out2
```

```
dloss_dw_out=(predicted_y-y)* sigmoid_derivative(predicted_y) * h_out
```

Code python

```
dloss_dw_out=hidden_layer_out.T.dot((predicted_y-y)*sigmoid_derivative(predicted_y))
```

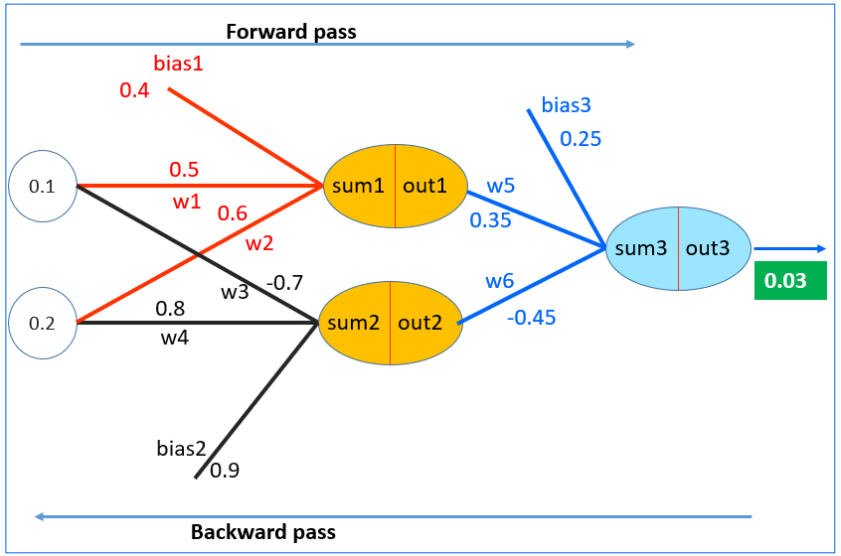
Update output layer bias

$$\frac{\partial Loss}{\partial bias_o}$$

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial Bias_out}$

$$\frac{\partial Loss}{\partial bias3} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial bias3}$$



$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

d_sum3_dbias3 =1

dloss_bias3=(predicted_y-y)* sigmoid_derivative(predicted_y) * 1

$$\frac{\partial Loss}{\partial bias_o}$$

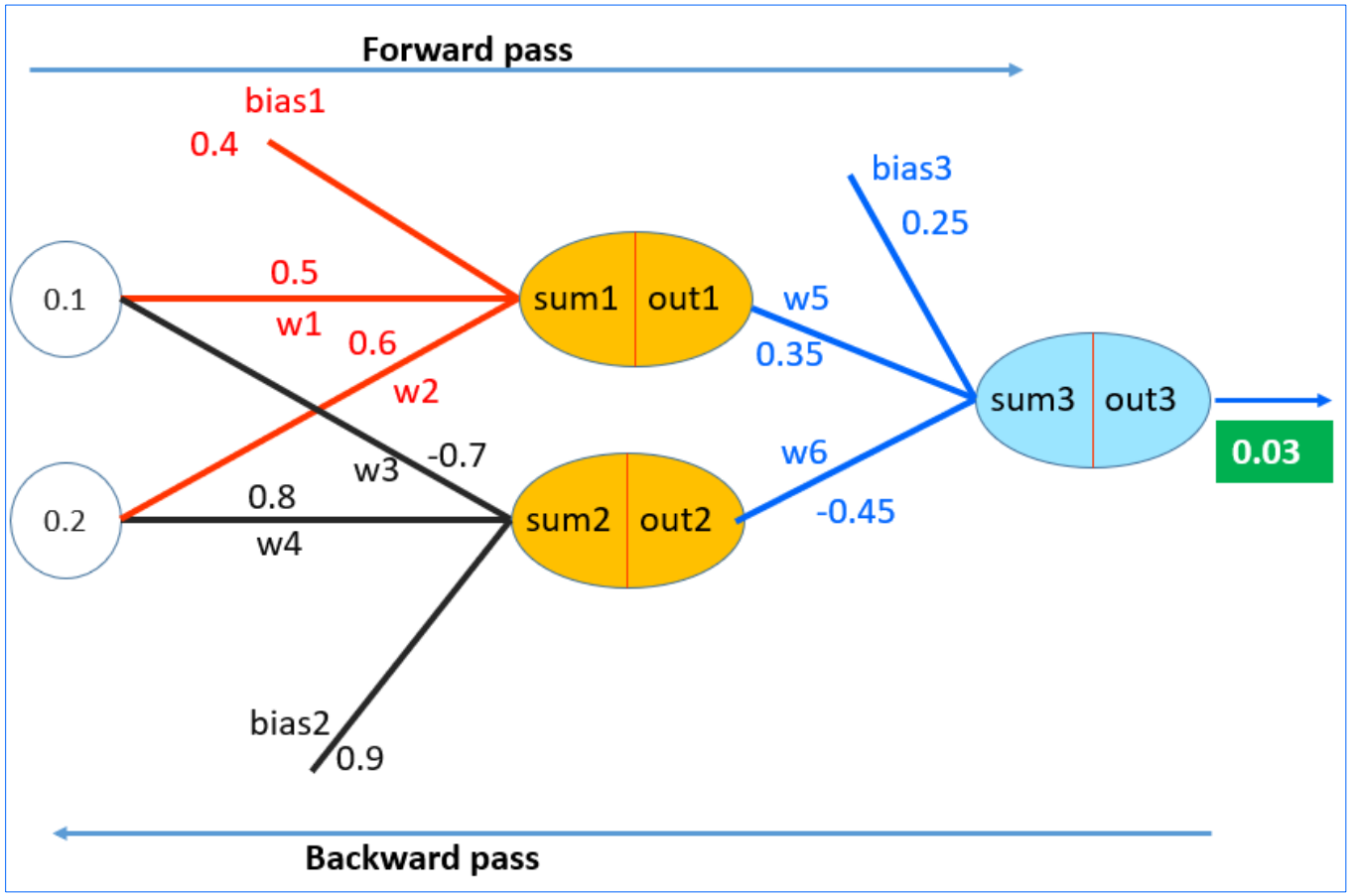
```
dloss_bias3=(predicted_y-y)* sigmoid_derivative(predicted_y) * 1
```

```
dloss_dbias_out=(predicted_y-y)* sigmoid_derivative(predicted_y)
```

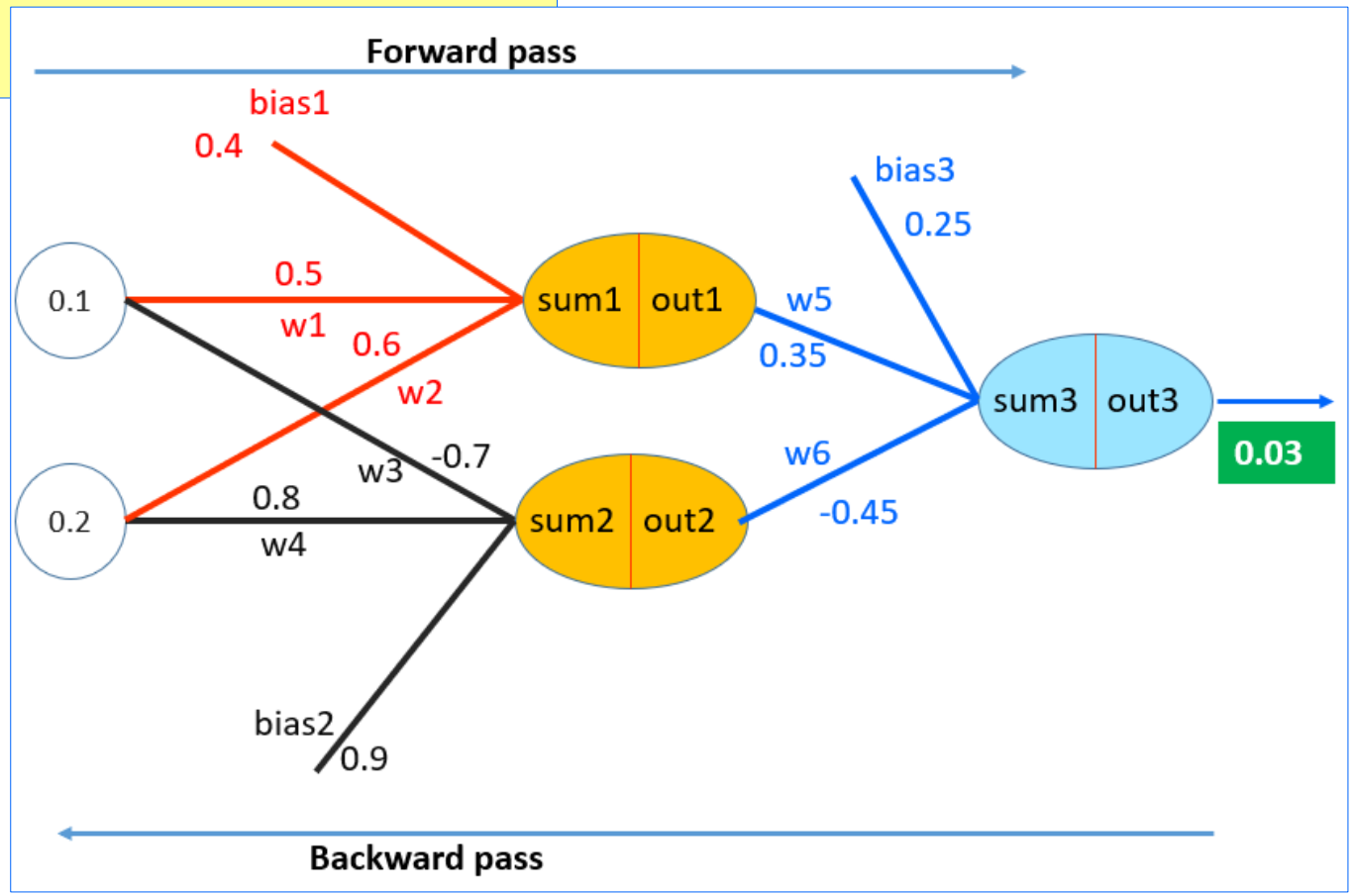
Code python

```
dloss_dbias_out = (predicted_y-y)*sigmoid_derivative(predicted_y)
```

```
#inputs
X = np.array([[0.1,0.2]])
y = np.array([[0.03]])
learning_rate= 0.1
#weights and bias initialization
hidden_weights = [[0.5,0.6],[-0.7,0.8]]
hidden_bias =[0.4,0.9]
output_weights = np.array([[0.35],[-0.45]])
output_bias = np.array([[0.25]])
```



```
#Learning process

#Forward Pass

hidden_layer_sum = np.dot(X,hidden_weights) + hidden_bias

hidden_layer_out = sigmoid(hidden_layer_sum)

output_layer_sum = np.dot(hidden_layer_out,output_weights) +
output_bias

predicted_y = sigmoid(output_layer_sum)
```



```
#Learning process

#Backward pass

#hidden layer

dloss_dw_h= X.T.dot((predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out))

dloss_dbias_h=(predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out)

#output layer

dloss_dw_out=hidden_layer_out.T.dot((predicted_y-y)*sigmoid_derivative(predicted_y))

dloss_dbias_out=(predicted_y-y)*sigmoid_derivative(predicted_y)
```

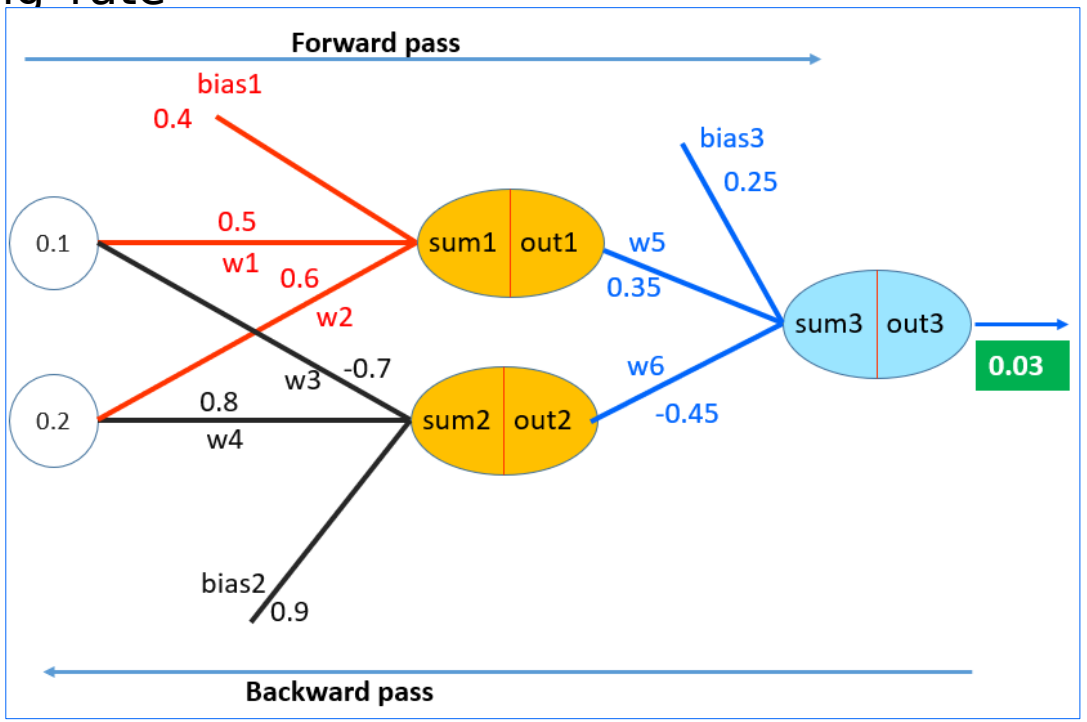
```
dloss_dbias_h=(predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out)
```


```
dloss_dbias_h=(predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out)
```


```
dloss_dw_out=hidden_layer_out.T.dot((predicted_y-y)*sigmoid_derivative(predicted_y))
```

```
dloss_dbias_out = (predicted_y-y)*sigmoid_derivative(predicted_y)
```

```
#update
hidden_weights-=dloss_dw_h*learning_rate
hidden_bias-=dloss_dbias_h*learning_rate
hidden_bias=np.sum(hidden_bias,axis=0,keepdims=True)
output_weights-=dloss_dw_out*learning_rate
output_bias-
=np.sum(dloss_dbias_out,axis=0,keepdims=True)*learning_rate
```



```
#print results
print("results-----------------------------------")
print("updated hidden weights: ")
print(hidden_weights)
print("updated hidden biases: ")
print(hidden_bias)
print("updated output weights: ")
print(output_weights)
print("updated output biases: ")
print(output_bias)
```

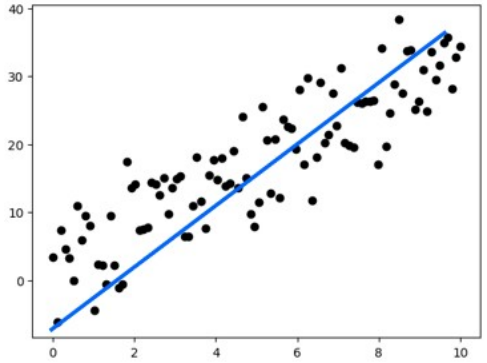
Questions

À faire:

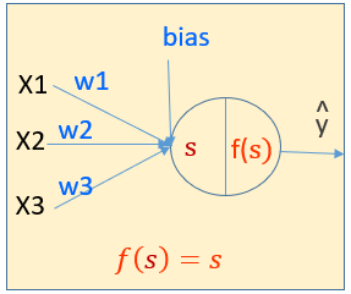
1. Compléter le code source pour traiter les deux problèmes qui restent:

1

Simple Linear Regression Problem



Random point cloud

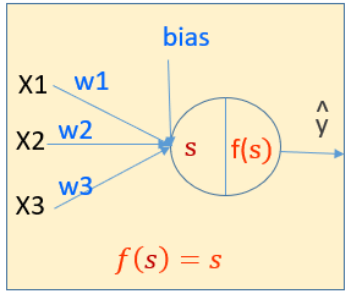


2

Multi Variate Linear Regression Problem

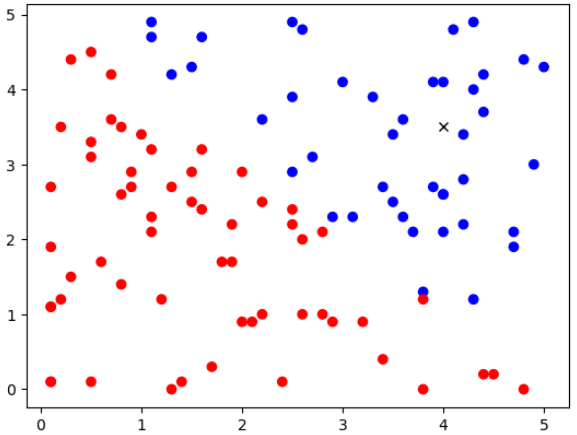
TV	radio	newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising p

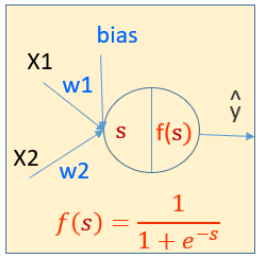


3

Linearly Separable problem

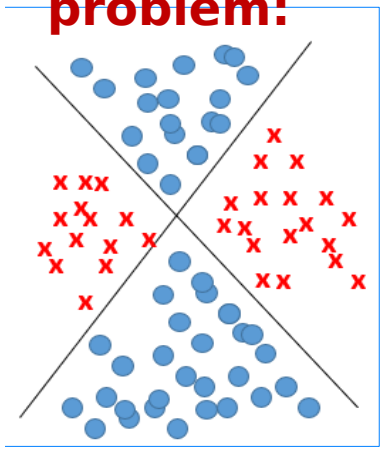


Red or Blue?

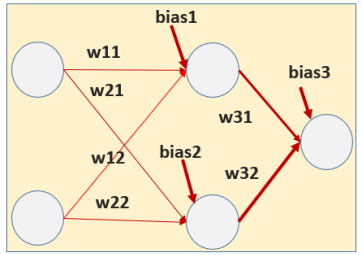


4

Non Linearly Separable problem:



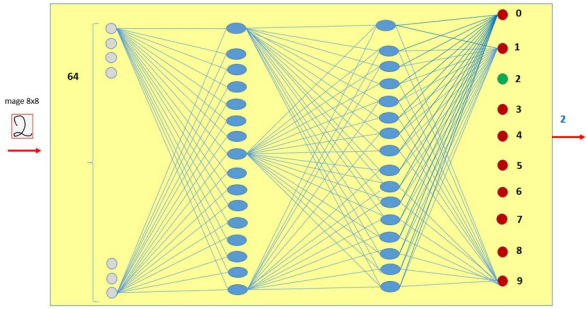
XOR Problem



5

Complex classification problem

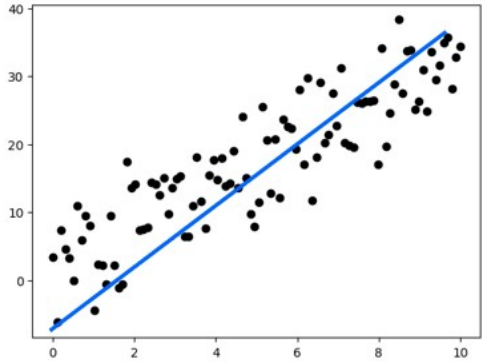
Hand Written digits



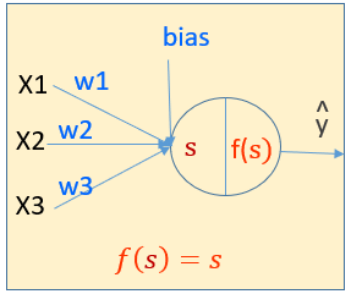
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

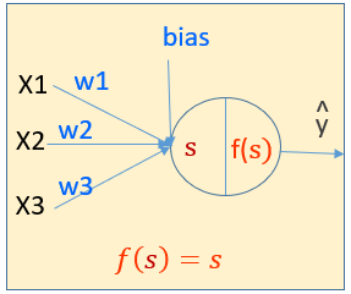


2

Multi Variate Linear Regression Problem

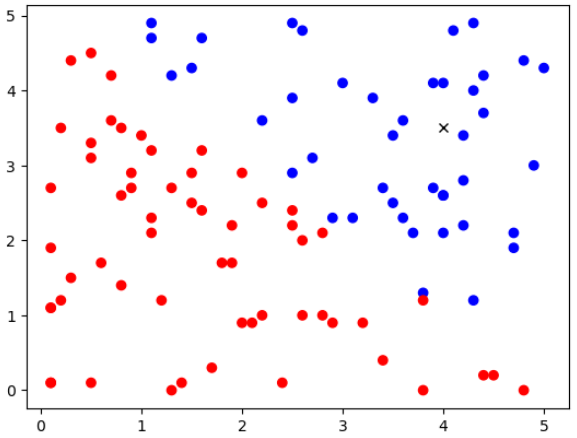
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising p

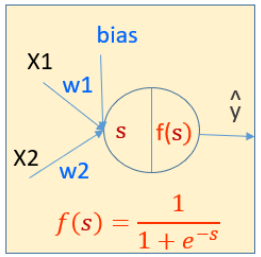


3

Linearly Separable problem

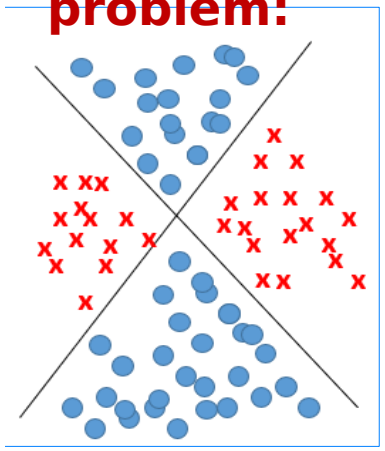


Red or Blue?

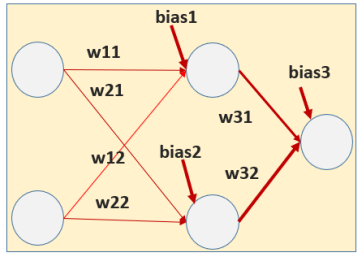


4

Non Linearly Separable problem:

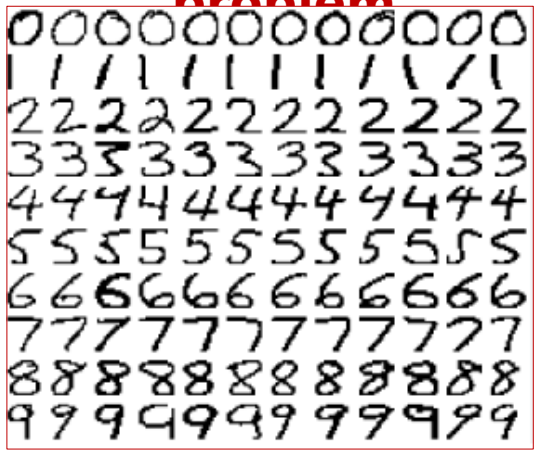
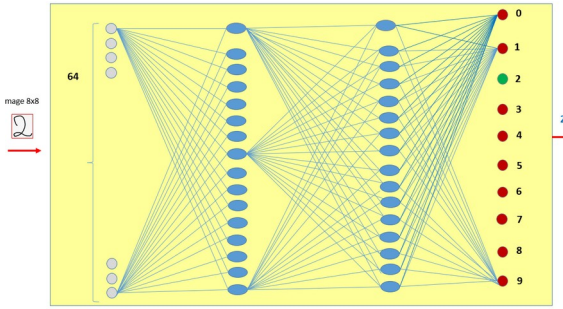


XOR Problem



5

Complex classification problem



Hand Written digits